FIAP ON

DIGITAL BUSINESS ENABLEMENT

JSF (INTERNACIONALIZAÇÃO, VALIDAÇÕES, NAVEGAÇÃO)

4

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 4.1 – JSF Request Response 10
Figura 4.2 – JSF MVC 11
Figura 4.3– JSF Request Response Renderer.....14
Figura 4.4 – JSF Ciclo de Vida.....16
Figura 4.5 – Configurar o projeto (I) 21
Figura 4.6 – Configurar o projeto(II) 22
Figura 4.7 – Configurando o projeto(III) 23
Figura 4.8– Configurando o projeto(IV) 23
Figura 4.9 – Configurar o projeto (V).....24
Figura 4.10 – Configurar o projeto(VI).....25
Figura 4.11 – Estrutura do projeto.....26
Figura 4.12 – Convert to Maven Project.....28
Figura 4.13 – POM Project.....28
Figura 4.14 – Create Java Class OlaMB 29
Figura 4.15 – Estrutura do Projeto 30
Figura 4.16 – Ola.xhtml 32
Figura 4.17 – Select Template 32
Figura 4.18 – Executar o Projeto.....37
Figura 4.19 – Testar o Projeto(I) 38
Figura 4.20 – Testar o Projeto(II) 38
Figura 4.21 – Testar o Projeto (III) 39
Figura 4.22 – New Dynamic Web Project.....47
Figura 4.23 – Configurar Project Facelets.....48
Figura 4.24 – Configurar Source Folders 48
Figura 4.25 – Configurar Web Modules.....49
Figura 4.26 – Configurar JSF Capabilities.....49
Figura 4.27 – Configurar o POM do Projeto 50
Figura 4.28 – A Estrutura do Projeto 51
Figura 4.29 – Recursos do Projeto.....62
Figura 4.30 – Os Templates do Projeto.....64
Figura 4.31 – Páginas de Negócio do Projeto 67
Figura 4.32 – Executar o Projeto.....74
Figura 4.33 – A tela index.xhtml em português 75
Figura 4.34 – A tela index.xhtml em inglês.....76
Figura 4.35 – A tela solicitacao.xhtml em inglês.....76
Figura 4.36 – A tela solicitacao.xhtml validada em inglês 77
Figura 4.37 – A tela solicitacao.xhtml com cadastro Ok em português 77

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 4.1 – Executar o Projeto 36
Quadro 4.2 – Bibliotecas JSF.....43

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 4.1 – web.xml 27
Código-fonte 4.2 – faces-config.xml 27
Código-fonte 4.3 – POM.xml 29
Código-fonte 4.4 – classe OlaMB 31
Código-fonte 4.5 – arquivo ola.xhtml 33
Código-fonte 4.6 – namespace http://xmlns.jcp.org/jsf/html 34
Código-fonte 4.7 – método saudar() 38
Código-fonte 4.8 – código fonte HTML 40
Código-fonte 4.9 – trecho da classe OlaMB 40
Código-fonte 4.10 – trecho da classe ChatMB 42
Código-fonte 4.11 – trecho do faces-config.xml 44
Código-fonte 4.12 – trecho da página solicitacao.xhtml 44
Código-fonte 4.13 – trecho da index.xhtml 45
Código-fonte 4.14 – trecho da solicitacao.xhtml 45
Código-fonte 4.15 – trecho da classe SolicitacaoBean 46
Código-fonte 4.16 – navegarHome da classe SolicitacaoBean 46
Código-fonte 4.17 – exemplo da navigation rule 47
Código-fonte 4.18 – POM do Projeto 50
Código-fonte 4.19 – classe Solicitacao.java 54
Código-fonte 4.20 – classe SolicitacaoBean.java 57
Código-fonte 4.21 – classe LinguagemBean.java 58
Código-fonte 4.22 – messages_en.properties 59
Código-fonte 4.23 – messages_pt_BR.properties 60
Código-fonte 4.24 – faces-config.xml 61
Código-fonte 4.25 – all.css 63
Código-fonte 4.26 – _template.xhtml 65
Código-fonte 4.27 – header.xhtml 66
Código-fonte 4.28 – context.xhtml 66
Código-fonte 4.29 – footer.xhtml 67
Código-fonte 4.30 – construcao.xhtml 68
Código-fonte 4.31 – trecho web.xml 68
Código-fonte 4.32 – index.xhtml 69
Código-fonte 4.33 – exemplo Regex 70
Código-fonte 4.34 – solicitação.xhtml 73

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

4 JSF (INTERNACIONALIZAÇÃO, VALIDAÇÕES, NAVEGAÇÃO).....7
4.1 JavaServer Faces 7
4.1.1 O modelo de programação em JSF 9
4.1.2 O JSF respondendo a uma requisição cliente.....10
4.1.3 JSF e o padrão MVC.....11
4.1.4 JSF e a camada Model.....12
4.1.5 JSF e a camada View.....12
4.1.6 JSF e a camada Controller.....13
4.1.7 Visão em alto nível do JSF e Facelets 14
4.1.8 O ciclo de vida JSF 16
4.1.9 Fase Restore View 17
4.1.10 Fase Apply Request Values 18
4.1.11 Fase Process Validations.....18
4.1.12 Fase Update Model Values 19
4.1.13 Fase Invoke Application 19
4.1.14 Fase Render Response.....20
4.2 Uma aplicação JSF de introdução.....20
4.2.1 Recursos necessários 20
4.2.2 Criar o Projeto no Eclipse.....21
4.2.3 A estrutura do Projeto.....26
4.2.4 O arquivo web.xml do Projeto 26
4.2.5 O arquivo de configuração faces-config.xml.....27
4.2.6 Configurando o Maven 28
4.2.7 O POM do Projeto 28
4.2.8 Criar um Bean Gerenciado.....29
4.2.9 Criar a página xhtml do Projeto 31
4.2.10 Ligar a página ao Managed Bean 34
4.2.11 Os componentes JSF e suas funções na página xhtml.....34
4.2.12 Executar o Projeto.....37
4.2.13 O código-fonte HTML retornado ao cliente.....39
4.3 Sobre o Managed Bean 40
4.4 Os escopos do JavaServer Faces 2.2.....41
4.4.1 O atributo eager 42
4.5 Bibliotecas de tags JSF.....42
4.6 Internacionalização 43
4.7 Navegação em JSF.....45
4.7.1 Navegação implícita (por links e botões).....45
4.7.2 Navegação implícita (por retorno de um método) 45
4.7.3 Navegação explícita 46
4.8 Praticar em JSF.....47
4.8.1 Criar o Projeto no Eclipse.....47
4.8.2 Criar o back-end da aplicação.....50
4.8.2.1 A classe entity Solicitacao.java 51
4.8.2.2 A classe Managed Bean SolicitacaoBean.java 54
4.8.2.3 A classe Managed Bean LinguangemBean.java 57
4.8.2.4 O arquivo de propriedades messages_en.properties.....58
4.8.2.5 O arquivo de propriedades messages_pt_BR.properties.....59

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.8.2.6 Configurar o faces-config.xml da aplicação 61
4.8.3 Criar o front-end da aplicação 61
4.8.3.1 Os arquivos de recursos da aplicação (img e css) 61
4.8.3.2 O arquivo css 62
4.8.3.3 Os templates da aplicação 63
4.8.3.3.1 A página _template.xtml 64
4.8.3.3.2 A página header.xtml 65
4.8.3.3.3 A página content.xtml 66
4.8.3.3.4 A página footer.xtml 66
4.8.3.4 As páginas que atendem ao negócio 67
4.8.3.4.1 A página construcao.xhtml 67
4.8.3.4.2 A página index.xhtml 68
4.8.3.4.3 A página solicitacao.xhtml 69
4.8.4 Executar o Projeto 74
4.9 Considerações finais 78
REFERÊNCIAS 79
GLOSSÁRIO 80

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 JSF (INTERNACIONALIZAÇÃO, VALIDAÇÕES, NAVEGAÇÃO)

4.1 JavaServer Faces

O *JavaServer Faces* (JSF) é um *framework* de interface do usuário (UI) para aplicativos da Web Java, fortemente baseado no padrão MVC. É projetado para facilitar significativamente o ônus de escrever e manter aplicações executadas em um Java Application Server.

JSF é parte da Plataforma Java Enterprise Edition e possui uma especificação Java formalizada padronizada pelo JCP (Java Community Process) que determina a construção das interfaces de usuário, baseadas em componentes.

A tecnologia JavaServer Faces consiste no seguinte:

- Uma API para representar componentes e gerenciar estado, para manipulação dos eventos, validação do lado do servidor, conversão dos dados, definição de navegação, apoio à internacionalização e à acessibilidade, fornecendo extensibilidade a todos esses recursos;
- Bibliotecas de tags para adicionar componentes às páginas da web e para conectar componentes aos objetos Java ao lado do servidor.

Utilizar componentes JSF permite que os desenvolvedores se concentrem mais na lógica dos negócios da aplicação e menos no desenvolvimento do design das páginas e nas ações, uma vez que o JavaScript e o CSS são abstraídos pelos componentes prontos.

O JSF fornece facilidade de uso das seguintes maneiras:

- Facilita a construção de uma UI a partir de um conjunto de componentes de UI pré-definidos e reutilizáveis;
- Simplifica a migração dos dados do aplicativo para a UI e da UI;
- Ajuda a gerenciar o estado da interface de usuário, em todos os pedidos ao servidor;
- Associa os eventos ao lado do cliente, com os manipuladores dos eventos ao lado do servidor;

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Permite que os componentes de UI personalizados sejam facilmente criados e reutilizados;
- Fornece separação das funções que envolvem a construção de aplicações Web;
- Utiliza Ajax em alguns dos seus componentes, tornando alguns processos mais rápidos e eficientes.

Na escrita deste material, a versão atual do JSF é a 2.3 para o Java EE 8, versão que inclui uma série de melhorias e aprimoramentos do Java 8, especificadas na (JSR 372: JavaServer Faces (JSF 2.3) Specification.), encontrada em https://jcp.org/en/jsr/detail?id=372. O JSF 2.3 funciona melhor com um full Java EE container. Não é o caso do Tomcat, portanto, para evitarmos a inclusão de várias dependências via Maven, utilizaremos o JSF 2.X neste material. Características do JSF 2.3 serão apresentadas em outro capítulo.

Existem duas implementações da especificação JSF, ambas desenvolvidas com licenças Open Source: a Oracle Mojarra e a Apache MyFaces. Neste material, utilizaremos a Oracle Mojarra.

Podemos dizer que um aplicativo JSF é semelhante a qualquer outro aplicativo web Java. É executado em um servlet container Java e, geralmente, contém:

- um conjunto de páginas da web nas quais os componentes são apresentados;
- bibliotecas de tags para representar manipuladores dos eventos, validadores e componentes UI;
- um conjunto de *managed beans*. Em uma aplicação JavaServer Faces, *managed beans* definem propriedades e funções para componentes UI em uma página;
- JavaBeans como modelos;
- classes auxiliares do lado do servidor e controladores para executar as ações do usuário;
- validadores, *handlers* dos eventos e da navegação;
- um arquivo descritor da implantação da web (web.xml);

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- opcionalmente, um ou mais arquivos da configuração do aplicativo, como o faces-config.xml, que pode ser usado para definir regras de navegação das páginas, configurar beans e outros objetos, como componentes personalizados;
- opcionalmente, um conjunto dos objetos personalizados, que podem incluir componentes personalizados, validadores, conversores ou *listeners*, criados pelos desenvolvedores;
- opcionalmente, um conjunto de tags personalizadas para representar objetos personalizados em páginas.

4.1.1 O modelo de programação em JSF

A tecnologia *JavaServer Faces* fornece um modelo de programação bem definido e várias bibliotecas de tags. As bibliotecas de tags contêm manipuladores que implementam as tags de componente.

Esses recursos aliviam significativamente o ônus de construção e manutenção de aplicativos da Web com interfaces de usuário (UIs) do lado do servidor. Com o mínimo esforço, é possível realizar as seguintes tarefas:

- criar uma página da Web;
- colocar componentes em uma página Web, adicionando-os por meio de tags;
- vincular, em uma página, componentes aos dados do lado do servidor;
- disparar, de componentes UI, o código do aplicativo do lado do servidor;
- salvar e restaurar o estado do aplicativo;
- reutilizar e personalizar componentes.

JSF é uma tecnologia *back-end*, ou seja, se você codificar um JSF e executá-lo diretamente em um navegador, não verá a representação gráfica esperada. Uma página JSF codificada em XHTML precisa ser processada pelo servidor para gerar um HTML, antes de ser enviado ao navegador.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Um desenvolvedor pode combinar componentes JSF com HTML e CSS, mas tudo isso será renderizado pelo servidor.

Uma página JSF pode ser vista como uma árvore dos componentes que usa várias bibliotecas de tags definidas pela especificação JSF, como o Mojarra, mas também bibliotecas dos terceiros, como o Primefaces, que veremos nos próximos capítulos.

O corpo da página é compilado em uma árvore de objetos gráficos Java e seus valores são vinculados a um objeto *bean* de suporte. Portanto, a página possui um ciclo de vida rico que lida com várias fases diferentes. Veremos o ciclo de vida, mais à frente.

4.1.2 O JSF respondendo a uma requisição cliente

A imagem abaixo mostra a interação entre um cliente e o servidor em um aplicativo *JavaServer Faces*. Em resposta a uma solicitação do cliente, uma página da Web é processada por um Web Container que implementa a tecnologia *JavaServer Faces.*

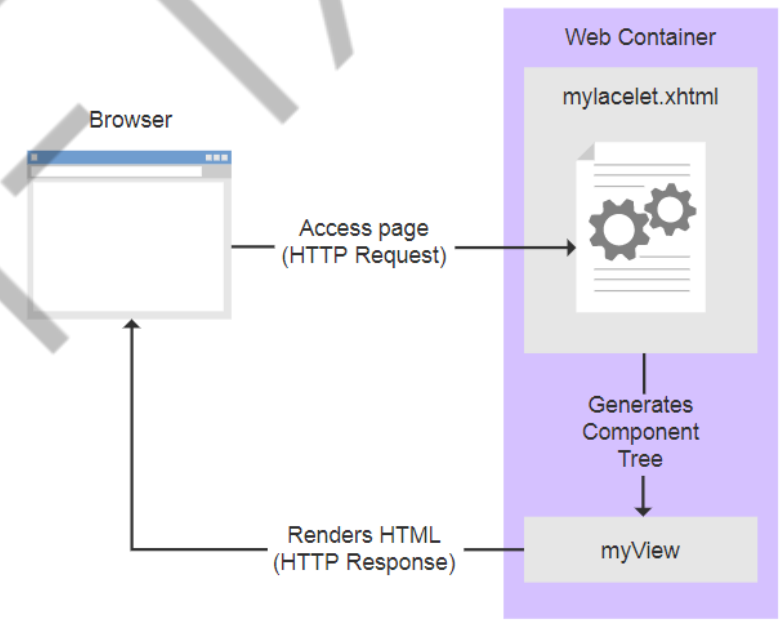

Figura 4.1 – JSF Request Response
Fonte: Oracle JSF (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Os containers provêm serviços de infraestrutura para execução dos componentes. Servlets, JSRs, páginas JSF e arquivos (HTML, css, imagens, etc.) necessitam de um Web Container para serem processados.

A página da Web myfacelet.xhtml é codificada usando as tags do componente *JavaServer Faces*. As tags são utilizadas para adicionar a árvore dos componentes ao view, que é a representação da página, ainda ao lado do servidor.

Além dos componentes, a página também pode fazer referência a outros objetos, tais como:

- *event listeners*, validadores e conversores registrados nos componentes;
- os componentes JavaBeans que capturam os dados e processam funcionalidades específica dos componentes.

Como resposta ao cliente, a exibição é renderizada. *Rendering* é o processo pelo qual o Web Container gera a resposta como HTML ou XHTML, para ser lida pelo cliente, geralmente um navegador.

4.1.3 JSF e o padrão MVC

JSF é um *server-side component framework* que utiliza o padrão de projeto MVC. Como sabemos, o padrão MVC separa o sistema em três camadas (Model, View e Controller).

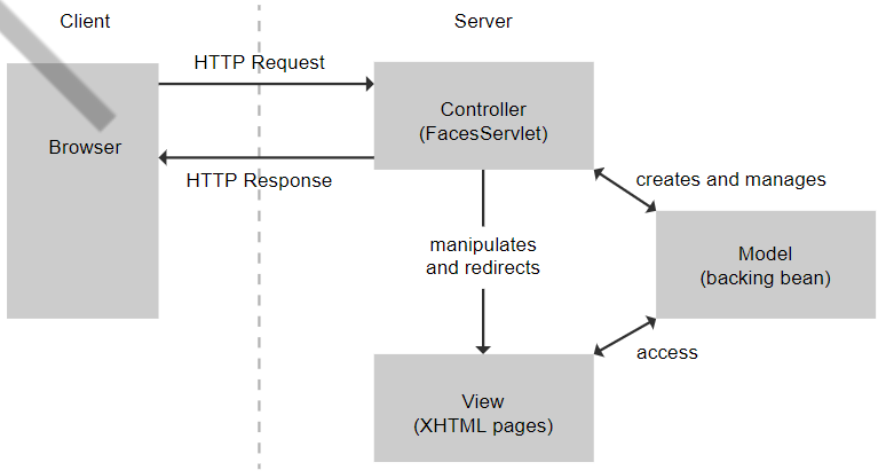

Figura 4.2 – JSF MVC
Fonte: Oracle JSF (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O MVC tem como objetivo manter dados e regras de negócios na camada Model, separadamente da camada View, que conterá as interfaces com usuário e da camada de Controller, que será responsável pelo fluxo da aplicação.

- **Model (backing bean)**
  - armazena o estado do aplicativo;
  - responde aos pedidos dos dados;
  - encapsula a lógica dos negócios.
- **View (páginas XHTML)**
  - renderiza a interface do usuário (UI);
  - solicita dados para a camada Model;
  - envia eventos a camada Model;
  - permite que o Controller selecione a próxima View.
- **Controller (FacesServlet)**
  - realiza o roteamento para a página correta;
  - mapeia as mudanças dos dados da UI para o Modelo;
  - passa dados da camada Model e para a camada Model.

4.1.4 JSF e a camada Model

Em JSF, o Model é composto por objetos de negócio Java e pela camada de dados do aplicativo, desenvolvida para trabalhar com classes *entity* (POJOs), com especial preferência por JPA e uma implementação, como o Hibernate.

4.1.5 JSF e a camada View

Em JSF, a camada de visão tem como tecnologia preferida e recomendada o uso de Facelets. O *Facelets* é uma linguagem de declaração poderosa e leve, que é usada para criar visualizações do *JavaServer Faces*, usando modelos de estilo HTML e para criar árvores dos componentes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Os recursos do Facelets incluem:

- uso do XHTML para criar páginas da web;
- suporte para bibliotecas de tags Facelets, além de JavaServer Faces e bibliotecas de tags JSTL;
- suporte para a linguagem de expressão (EL);
- templação para componentes e páginas.

As visualizações de Facelets geralmente são criadas como páginas XHTML. As implementações do *JavaServer Faces* suportam páginas XHTML criadas de acordo com a Definição de Tipo de Documento Transicional (DTD).

Por convenção, as páginas web construídas com XHTML possuem a extensão .xhtml.

A tecnologia *JavaServer Faces* oferece suporte às várias bibliotecas de tags que nos possibilitam adicionar componentes a uma página Web. Para suportar o mecanismo da biblioteca de tags JSF, o Facelets usa declarações de *namespace* XML.

4.1.6 JSF e a camada Controller

Em JSF, a camada *controller* é executada por uma *servlet* chamada Faces Servlet ou opcionalmente, por arquivos de configuração em XML ou por *event handlers* e *listeners*.

Todas as requisições dos usuários passam pela *Faces Servlet*, que examina cada requisição e dispara uma ação no modelo, usando *managed beans*.

A *Faces Servlet* é interna e parte do JSF. Tem como única maneira de configuração o uso dos metadados externos. Até a JSF 1.2, a única fonte de configuração era o arquivo faces-config.xml.

Atualmente, com JSF 2.x, esse arquivo tornou-se opcional e a maioria dos metadados pode ser definido através das anotações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.1.7 Visão em alto nível do JSF e Facelets

Resumidamente, as *Faces Servlet* recebem todas as requisições da view cliente, realizam uma navegação entre views ou disparam uma ação no model, recebem o processamento do negócio e enviam uma resposta ao cliente.

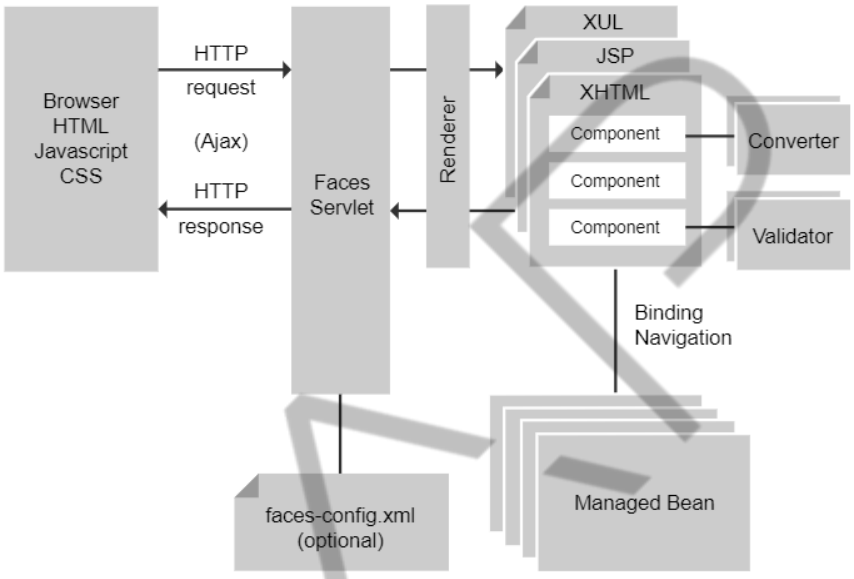

Figura 4.3– JSF Request Response Renderer
Fonte: Oracle JSF (2018)

O diagrama apresenta várias peças importantes do JSF que tornam sua arquitetura rica e flexível:

- **FacesServlet e faces-config.xml**: Faces Servlet é o *servlet* principal para o aplicativo e pode ser configurado opcionalmente por um arquivo descritor faces-config.xml.
- **Páginas com componentes**: Utilize Facelets (que é o padrão recomendado a partir do JSF 2.0).
- **Renderers**: São responsáveis por exibir um componente e traduzir uma entrada do usuário nos valores da propriedade do componente.
- **Converters**: Convertem o valor de um componente.
- **Validators**: São responsáveis por garantir que o valor inserido por um usuário em um componente seja válido.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Managed bean e navegação**: A lógica dos negócios é codificada em beans gerenciados, que também controlam a navegação entre as páginas do aplicativo.
- **Suporte do Ajax**: o JSF 2.3 possui suporte interno para o Ajax.

Entendendo como um aplicativo JSF com Facelets funciona:

1. Quando um cliente, como um navegador, faz uma nova solicitação para uma página criada por *Facelets*, uma nova árvore dos componentes é criada colocada no *FacesContext.*
2. A nova árvore é aplicada aos *Facelets* e a exibição é preenchida com componentes para renderização.
3. A visão recém-construída é retornada como uma resposta ao cliente.
4. Na renderização, o estado dessa exibição é armazenado para o próximo pedido. O estado dos componentes de entrada e os dados do formulário são armazenados.
5. O cliente pode interagir com a visualização e solicitar outra e/ou alterar a partir do aplicativo *JavaServer Faces*. Nesse momento, a view salva é restaurada a partir do estado armazenado.
6. A visão restaurada é mais uma vez passada pelo ciclo de vida do *JavaServer Faces*, que, eventualmente, gerará uma nova visualização ou renderizará novamente a exibição atual, se não houver problemas de validação e nenhuma ação for acionada.
7. Se a mesma visualização for solicitada, a exibição armazenada será renderizada novamente.
8. Se uma nova visualização for solicitada, o processo descrito no passo 2 será continuado.
9. A nova visão é então retornada como uma resposta ao cliente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.1.8 O ciclo de vida JSF

O ciclo de vida de uma aplicação refere-se às várias etapas do processamento de um aplicativo JSF, desde a sua iniciação até a conclusão. Todas as aplicações têm ciclos de vida. Durante o ciclo de vida de uma aplicação web, tarefas comuns são realizadas, entre elas:

- processar as requisições recebidas;
- decodificar parâmetros recebidos;
- modificar e salvar o estado;
- renderizar páginas web para o navegador, como resposta.

O *framework* JSF administra as fases do ciclo de vida automaticamente para aplicativos simples ou permite que nós o façamos manualmente, sempre que necessário, geralmente em casos das aplicações complexas.

O ciclo de vida de um aplicativo *JavaServer Faces* começa quando o cliente faz uma solicitação HTTP para uma página e termina quando o servidor responde com a página, em HTML.

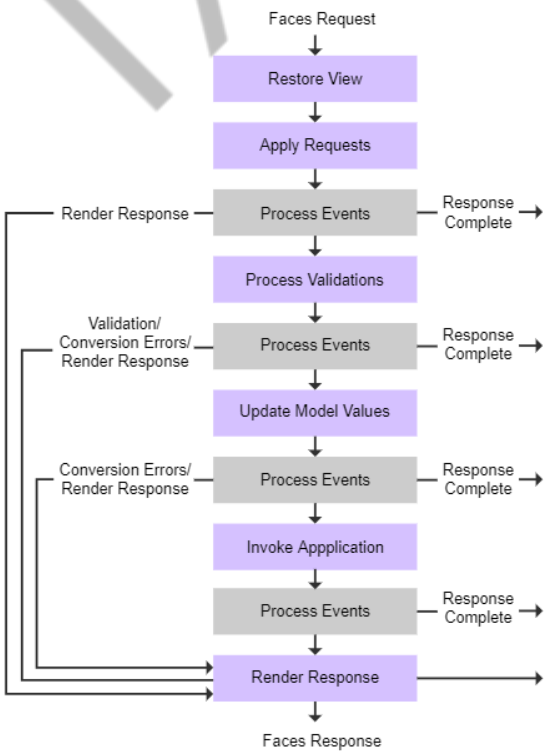

Figura 4.4 – JSF Ciclo de Vida
Fonte: Oracle JSF (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como já visto, uma página JSF é constituída por uma árvore de componentes. Durante o ciclo de vida, a implementação *JavaServer Faces* deve criar a view considerando, sempre que existente, o estado salvo de uma submissão anterior da página.

Todas as vezes que o cliente solicita uma página, a implementação JSF executa várias tarefas, como validar a entrada dos dados dos componentes e converter aos dados de entrada para tipos especificados ao lado do servidor.

O ciclo de vida *Request-Response* tem dois tipos de solicitações: iniciais e postbacks. Uma solicitação *inicial* ocorre quando o usuário faz a primeira solicitação por uma página. A outra, de *postback*, ocorre quando o usuário envia os dados de um formulário contido em uma página carregada no navegador, obtida em uma solicitação inicial.

Quando o ciclo de vida desenvolve a solicitação inicial, executa apenas as fases *Restore View e Render Response*. Por outro lado, quando o ciclo de vida administra um postback, executa todas as fases do ciclo.

4.1.9 Fase Restore View

Esta fase se inicia quando a solicitação para uma página JSF é feita, geralmente pela ação de navegação em um link ou um componente button é clicado.

Durante esta fase, a implementação do JSF cria a visão da página, com manipuladores dos eventos e validadores para componentes e salva a exibição na *FacesContext,* que contém todas as informações necessárias para processar uma única solicitação. Todos os componentes, manipuladores dos eventos, conversores e validadores do aplicativo têm acesso à *instância* de um *FacesContext*.

Se a solicitação para a página for um pedido inicial, a implementação do *JavaServer Faces* cria, nesta fase, uma exibição vazia e o ciclo de vida avança para a fase *Render Response*, onde a exibição vazia será preenchida com os componentes referenciados pelas tags codificados na página.

Se o pedido para a página for um *postback*, uma exibição correspondente a essa página existe na *instância FacesContext*. Durante esta fase, a implementação

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

JSF restaura a exibição usando as informações de estado salvas no cliente ou no servidor.

4.1.10 Fase Apply Request Values

Depois que a árvore do componente é restaurada, cada componente na árvore extrai seu novo valor dos parâmetros contidos na requisição. O valor é armazenado localmente, em cada um deles.

Se algum método ou *event listener* chamou o método *Render Response* na instância *FacesContext* atual, a implementação do JSF salta para a fase de *Render Response.*

Se algum evento tiver sido enfileirado durante essa fase, a implementação *JavaServer Faces* transmite os eventos para os Listeners interessados.

Se alguns componentes na página tiverem seus atributos *immediate* como *true*, as validações, conversões e eventos associados a esses componentes serão processados durante esta fase.

Se alguma conversão falhar, uma mensagem de erro associada ao componente é gerada e colocada na fila *FacesContext*. Essa mensagem será exibida durante a fase *Render Response*, juntamente com quaisquer erros de validação resultantes da fase *Process Validations*.

No final dessa fase, os componentes são definidos para seus novos valores, e mensagens e eventos estão enfileirados.

Se a solicitação atual for identificada como uma solicitação parcial, o contexto parcial será recuperado *FacesContext* e o método de processamento parcial será aplicado.

4.1.11 Fase Process Validations

Durante esta fase, a implementação do JSF processa todos os validadores registrados nos componentes da árvore. Examina os atributos do componente que

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

possuem regras para a validação e aplica essas regras ao valor armazenado para o componente.

A implementação *JavaServer Faces* também conclui conversões para componentes de entrada que não possuem o atributo *immediate* definido como *true.*

Se o valor local for inválido ou se alguma conversão falhar, a implementação do *JavaServer Faces* adiciona uma mensagem de erro à *FacesContext* e o ciclo de vida avança diretamente para a fase de *Render Response,* para que a página seja renderizada novamente com as mensagens de erro exibidas junto com erros de conversão da fase *Apply Request Values*, quando existentes.

Se os eventos tiverem sido enfileirados durante esta fase, a implementação JSF os transmite para ouvintes interessados.

4.1.12 Fase Update Model Values

Após determinar que os dados sejam válidos, a implementação JSF atualiza apenas as propriedades *bean* apontadas pelo atributo value de um componente de entrada.

Se os dados locais não puderem ser convertidos para os tipos especificados pelas propriedades do *bean*, o ciclo de vida avança diretamente para a fase *Render Response* para que a página seja renderizada com erros exibidos, semelhante ao que acontece quando há erros de validação.

Se algum evento tiver sido enfileirado durante esta fase, a implementação *JavaServer Faces* os transmite para ouvintes interessados.

4.1.13 Fase Invoke Application

Durante esta fase, a implementação *JavaServer Faces* lida com todos os eventos de negócio e navegação do aplicativo, como enviar um formulário para cadastro ou navegar em um link para outra página.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Neste ponto, se o aplicativo precisar redirecionar para um recurso de aplicativo da Web diferente ou gerar uma resposta que não contenha componentes JSF, pode chamar o método *FacesContext.responseComplete.*

Se a view que está sendo processada foi reconstruída a partir das informações de estado de uma solicitação anterior, e se um componente disparou um evento, esses são transmitidos para ouvintes interessados.

Finalmente, a implementação *JavaServer Faces* transfere o controle para a fase *Render Response*.

4.1.14 Fase Render Response

Durante esta fase, o *JavaServer Faces* cria a view e delega autoridade para o recurso apropriado para renderizar as páginas.

Se for um pedido inicial, os componentes representados na página serão adicionados à árvore do componente. Se não for um pedido inicial, os componentes estão adicionados à árvore e não precisam ser somados novamente.

Se a solicitação for um *postback* e erros foram encontrados durante a fase de *Apply Request Values, Process Validations, or Update Model Values*, a página original é renderizada novamente durante esta fase.

Se as páginas contêm tags de objetos para exibição de mensagem, todas as mensagens de erro contidas na fila serão exibidas na página.

Após o conteúdo da exibição ser renderizado, o estado da resposta é salvo para que as solicitações subsequentes possam acessá-lo. O estado salvo está disponível para a fase de Restore View.

4.2 Uma aplicação JSF de introdução

4.2.1 Recursos necessários

- JDK 8
- Eclipse

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Tomcat 8
- Mojarra

4.2.2 Criar o Projeto no Eclipse

No Eclipse, crie um novo projeto via wizard.

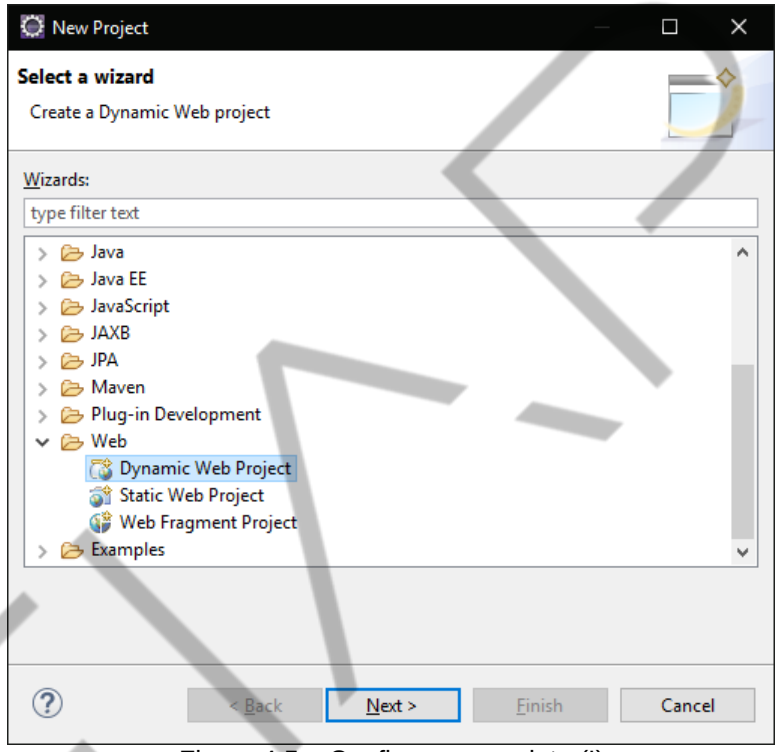

Figura 4.5 – Configurar o projeto (I)
Fonte: Elaborado pelo autor (2018)

No Eclipse, selecione File > New > Dynamic Web Project e configure-o, conforme a imagem. Clique em Modify.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

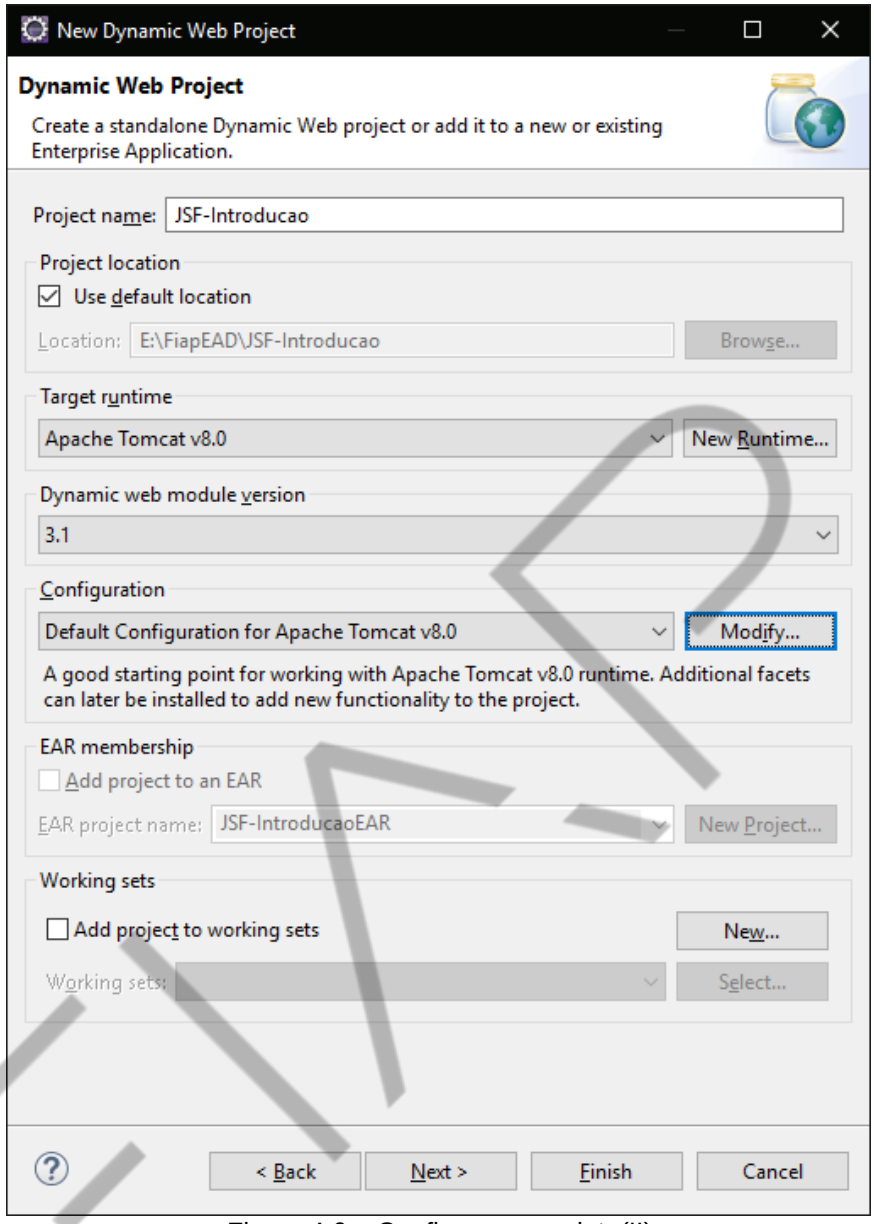

Figura 4.6 – Configurar o projeto(II)
Fonte: Elaborado pelo autor (2018)

Em Project Facets, selecione *Java Server faces* e clique OK.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

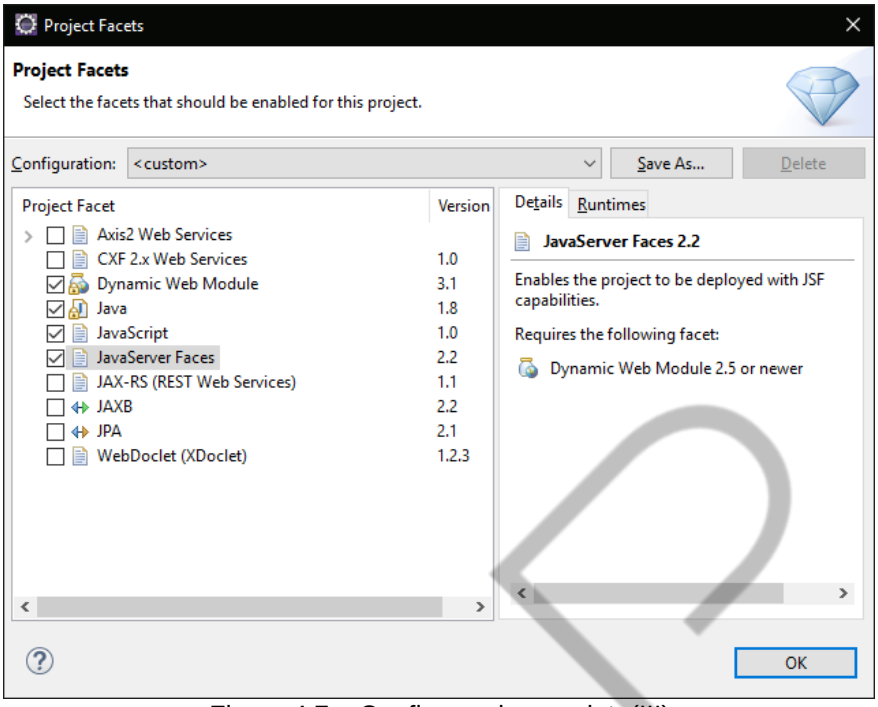

Figura 4.7 – Configurando o projeto(III)
Fonte: Elaborado pelo autor (2018)

Clique em Next.

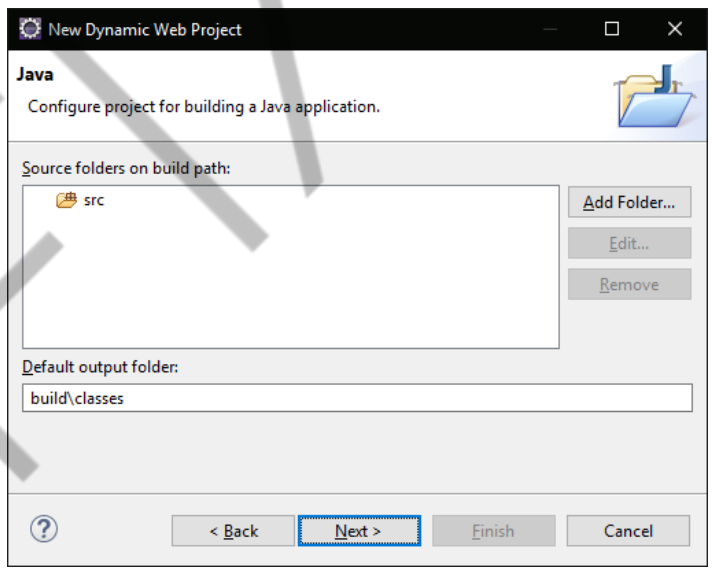

Figura 4.8 – Configurando o projeto(IV)
Fonte: Elaborado pelo autor (2018)

Em Web Module, selecione Generate web.xml.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

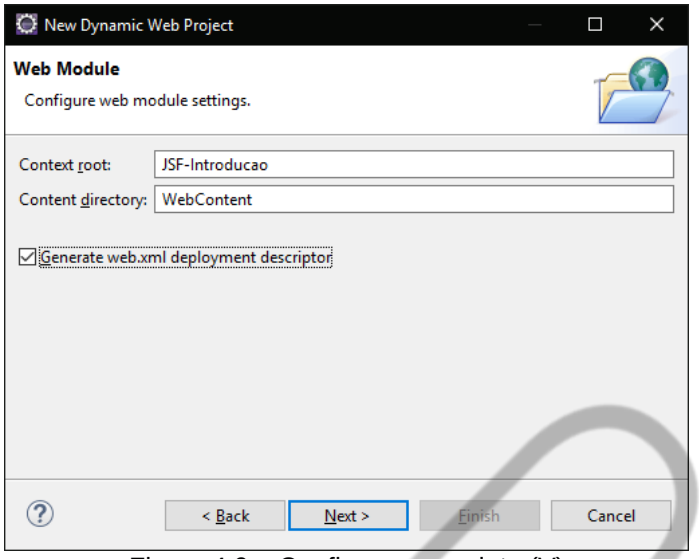

Figura 4.9 – Configurar o projeto (V)
Fonte: Elaborado pelo autor (2018)

O *Content root* é o diretório do aplicativo com maior nível, quando for implantado em um servidor Web. Essa configuração resultará no seguinte link para acesso à aplicação: http://localhost:8080/JSF-Introducao/

O *Content directory* é uma pasta que será criada na estrutura do projeto, na qual, obrigatoriamente, todos os recursos do aplicativo Web deverão ser armazenados. Por exemplo, páginas, folhas de estilo ou imagens devem ser armazenadas nesse diretório ou em subdiretórios, caso isso não ocorra, não estarão disponíveis para acesso pelo cliente, quando o aplicativo for executado no servidor.

O *deployment descriptor* ou web.xml é um arquivo de configuração que descreve como implantar um módulo no servidor, especificando configurações e opções do container.

Selecionamos o *checkbox* para que o arquivo web.xml seja gerado. Nesse arquivo, vamos configurar o container Web do Tomcat para que execute o *servlet FacesServlet,* responsável por administrar o ciclo de vida da nossa aplicação JSF e inicializar os recursos exigidos pela tecnologia no servidor. Clique em Next.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em JSF Capabilities, configure o *wizard* conforme a imagem abaixo.

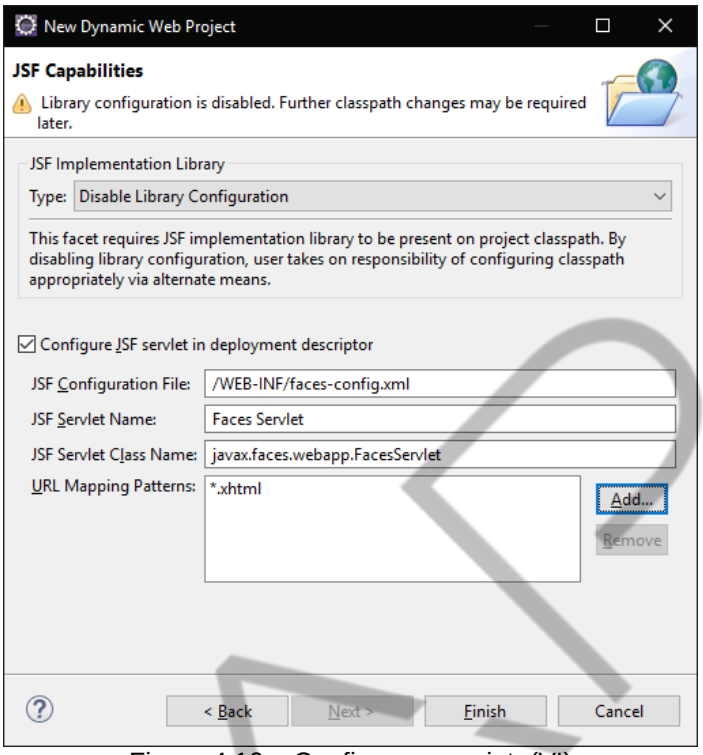

Figura 4.10 – Configurar o projeto(VI)
Fonte: Elaborado pelo autor (2018)

Deixe JSF *Implementation Library* como *Disable*, pois o *Maven* fará o trabalho de configurar o *classpath* com a biblioteca do Mojarra.

Para a nossa aplicação JSF, essa tela do *wizard* criará um arquivo chamado faces-config.xml e registrará no arquivo web.xml, a *servlet Faces Servlet*. Todas as solicitações para o nosso aplicativo, que referenciem componentes JSF, passarão por javax.faces.webapp.FacesServlet.

O mapeamento para *FacesServlet* pode ser um mapeamento de prefixo, como /faces/*, ou um mapeamento de extensão, como *.xhtml.

Como optamos pelo mapeamento de extensão, se uma solicitação chegar ao servidor para uma página da nossa aplicação com extensão. xhtml, o container Web do Tomcat enviará tal solicitação para a instância do *FacesServlet*, para que a página seja processada. Clique em *Finish.*

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2.3 A estrutura do Projeto

O Projeto JSF-Introdução será criado, com a seguinte estrutura:

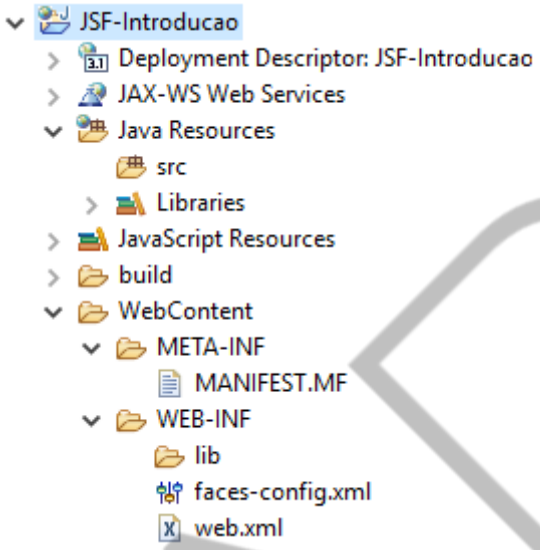

Figura 4.11 – Estrutura do projeto
Fonte: Elaborado pelo autor (2018)

4.2.4 O arquivo web.xml do Projeto

Arquivo web.xml gerado.

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app
     xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/jav
aee
    http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
    id="WebApp_ID" version="3.1">

  <display-name>JSF-Introducao</display-name>

  <servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>
         javax.faces.webapp.FacesServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        <servlet-mapping>
          <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>*.xhtml</url-pattern>
  </servlet-mapping>

</web-app>
```

Código-fonte 4.1 – web.xml
Fonte: Elaborado pelo autor (2018)

Como você sabe, por padrão, uma *servlet* é carregada pelo container no primeiro pedido que receba. A configuração <load-on-startup> no web.xml faz com que a *FacesServlet* da nossa aplicação seja carregada no momento da implantação do projeto ou no início do Tomcat. Dessa forma, demorará menos tempo para responder ao primeiro pedido que a nossa aplicação receber. O número 1 determina a prioridade de carregamento.

4.2.5 O arquivo de configuração faces-config.xml

O faces-config.xml gerado.

```
<?xml version="1.0" encoding="UTF-8"?>
<faces-config
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation=
 "http://xmlns.jcp.org/xml/ns/javaee
  http://xmlns.jcp.org/xml/ns/javaee/web-facesconfig_2_2.xsd"
  version="2.2">

</faces-config>
```

Código-fonte 4.2 – faces-config.xml
Fonte: Elaborado pelo autor (2018)

No JSF 1.0 o arquivo faces-config.xml era obrigatório nas aplicações *JavaServer Faces*, a partir do JSF 2.0 (versão que suporta anotações), o arquivo faces-config.xml deixou de ser obrigatório e passou a ser utilizado apenas quando for necessário fazer configurações da aplicação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2.6 Configurando o Maven

Agora, vamos configurar o nosso projeto do Eclipse para utilizar o Maven. Com o botão direito sobre o projeto, converta-o para Maven Project.

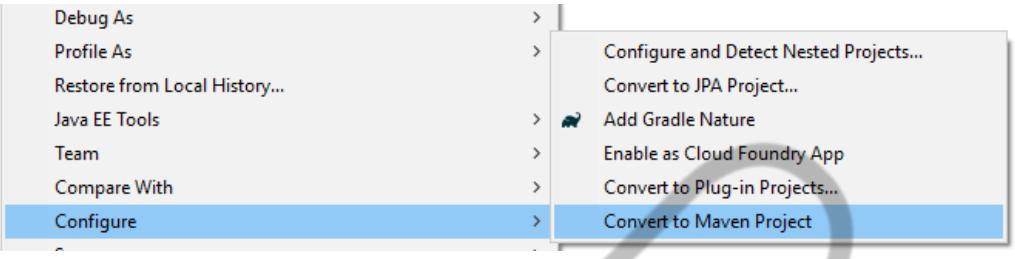

Figura 4.12 – Convert to Maven Project
Fonte: Elaborado pelo autor (2018)

4.2.7 O POM do Projeto

Configure o Maven POM.

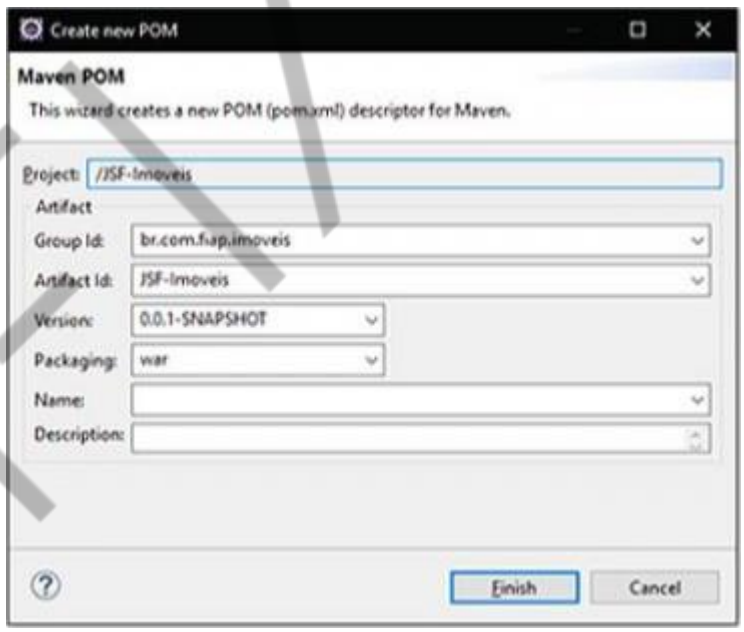

Figura 4.13 – POM Project
Fonte: Elaborado autor (2018)

Inclua a dependência abaixo de *war*, neste exemplo introdutório, utilizaremos a versão 2.2.2 do Mojarra.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<dependencies>
    <!-- Mojarra 2.2.2 -->
    <dependency>
        <groupId>org.glassfish</groupId>
        <artifactId>javax.faces</artifactId>
        <version>2.2.2</version>
        <scope>compile</scope>
    </dependency>
</dependencies>
```

Código-fonte 4.3 – POM.xml
Fonte: Elaborado pelo autor (2018)

4.2.8 Criar um Bean Gerenciado

Em src, crie uma Nova Classe Java.

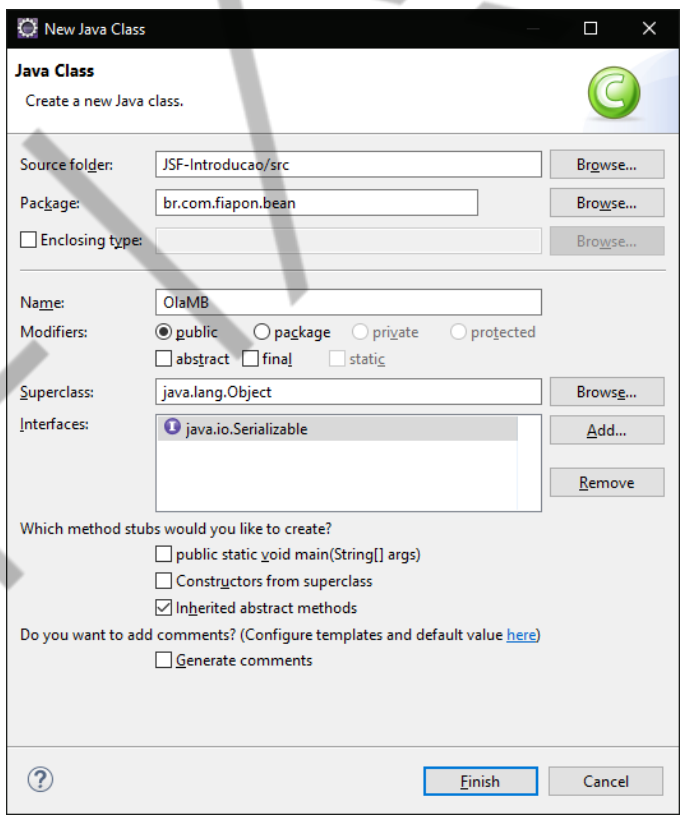

Figura 4.14 – Create Java Class OlaMB
Fonte: Elaborado pelo autor (2018)

O projeto ficará com a seguinte estrutura de pacote.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

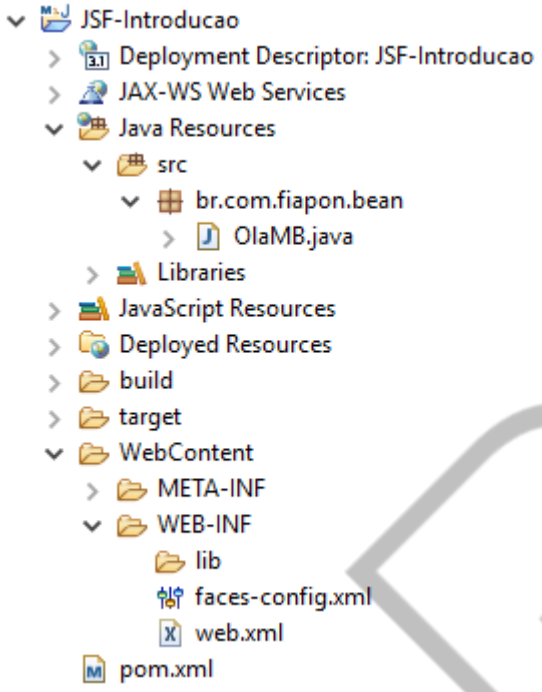

Figura 4.15 – Estrutura do Projeto
Fonte: Elaborado pelo autor (2018)

Codifique a classe OlaMB.

```
package br.com.fiapon.bean;

import java.io.Serializable;
import javax.faces.bean.ManagedBean;

@ManagedBean
public class OlaMB implements Serializable{


    private static final long serialVersionUID = 1L;
    private String nome;
    private String resposta;

    public void saudar() {
        System.out.println(nome);
        resposta = "Olá! " + nome;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        public String getResposta() {
            return resposta;
        }

        public void setResposta(String resposta) {
            this.resposta = resposta;
        }
    }
```

Código-fonte 4.4 – classe OlaMB
Fonte: Elaborado pelo autor (2018)

A anotação @ManagedBean registra a classe OlaMB como um JSF Managed Bean. Um Managed Bean é uma classe Java Bean gerenciada pelo JSF que torna possível a interação entre a interface do usuário e a lógica de negócios em uma aplicativa web JavaServer Faces.

As principais funções de um Managed Bean são receber, no servidor, os dados digitados pelo usuário, processar ou chamar objetos de negócio para processá-los e fornecer os dados para apresentação na UI.

Como vimos, o JSF é um *framework* MVC, portanto, a camada View formada pelos componentes da página ficará separada da Model, camada que implementa os comportamentos da página.

A UI da nossa aplicação será codificada na página ola.xhtml e, por ser uma aplicação muito simples, a regra de negócio será executada no método saudar, da classe Managed Bean OlaMB.

São boas práticas padronizar os nomes das páginas sempre com letras minúsculas e não utilizar acentos ou espaços nos nomes. Também devemos padronizar os nomes dos nossos ManagedBeans com os sufixos Bean ou MB.

Neste material, utilizaremos um exemplo de cada extensão.

4.2.9 Criar a página xhtml do Projeto

Em WebContent, botão direito, crie um novo arquivo HTML, renomeie a extensão para xhtml.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

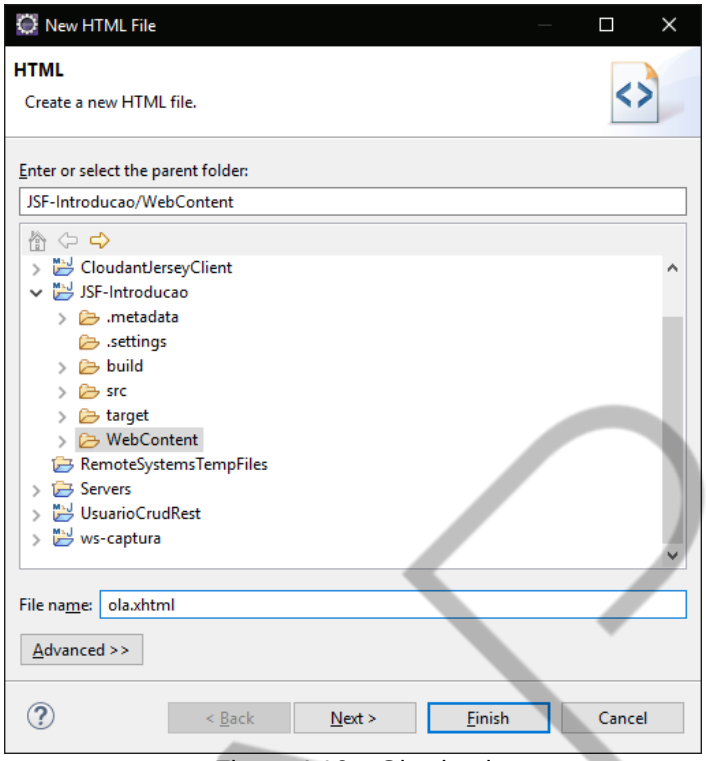

Figura 4.16 – Ola.xhtml
Fonte: Elaborado pelo autor (2018)

Clique em Next e selecione o Template html 5.

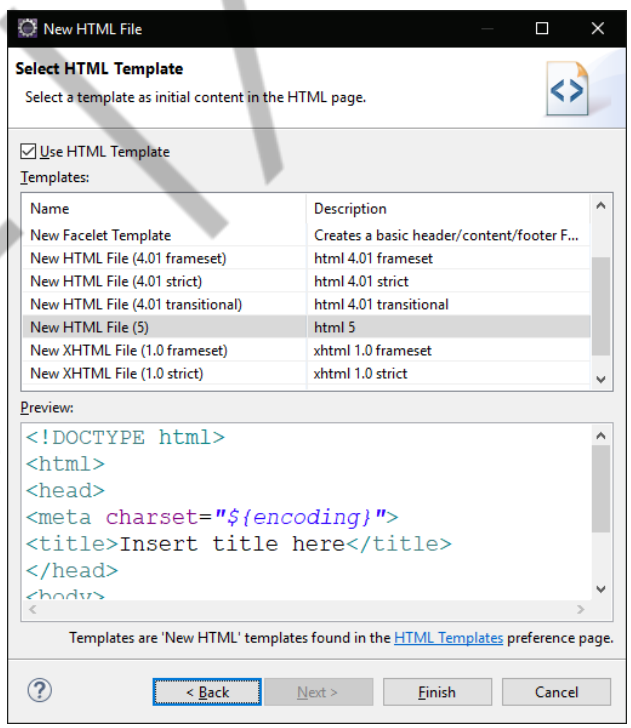

Figura 4.17 – Select Template
Fonte: Elaborado pelo autor (2018)

Codifique o arquivo ola.xhtml.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html">

<h:head>
<title>Seja bem vindo!</title>
</h:head>
<h:body>

    <h1>Bem vindo ao mundo JSF!</h1>

    <h:form id="saudacaoForm" >
         <h:panelGrid id="formTable"
            columns="1" cellpadding="5">


            <h:outputLabel id="nomeLabel"
                  value="Digite seu nome:"
                for="nomeInput" />

              <h:inputText id="nomeInput"
                  value="#{olaMB.nome}" />

              <h:commandButton id="saudarButton"
                  value="Diga olá"
                              type="submit"
            action="#{olaMB.saudar}" />

              <h:outputText id="respostaOutput"
                  value="#{olaMB.resposta}" />

         </h:panelGrid>
    </h:form>

</h:body>
</html>
```

Código-fonte 4.5 – arquivo ola.xhtml
Fonte: Elaborado pelo autor (2018)

A declaração <! DOCTYPE html> não é uma tag HTML, é uma instrução para o navegador saber a versão do HTML em que a página está escrita. Como queremos HTML 5, codificamos apenas <! DOCTYPE html>.

A tecnologia *JavaServer Faces* oferece suporte a várias bibliotecas de tags para adicionar componentes a uma página Web. Para suportar o mecanismo de biblioteca de tags JSF, devemos incluir na nossa página uma declaração de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

namespace XML. Em nosso código, acessamos a biblioteca de componentes HTML através do prefixo `h:` com o namespace http://xmlns.jcp.org/jsf/html.

```
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="http://xmlns.jcp.org/jsf/html">
```

Código-fonte 4.6 – namespace http://xmlns.jcp.org/jsf/html
Fonte: Elaborado pelo autor (2018)

4.2.10 Ligar a página ao Managed Bean

Após o *Java Bean OlaMB* se tornar gerenciável pelo JSF, através da anotação *@ManagedBean*, seus atributos (via getters e setters) e métodos podem ser acessados pela página ola.xhtml, utilizando EL (Expression Language).

Em nossa aplicação, utilizamos os componentes JSF contidos na tabela abaixo e ligamos alguns deles ao *Managed Bean*, através das expressões EL contidas entre os caracteres #{ }. Para acessar um *Managed Bean*, precisamos invocá-lo sempre com a primeira letra minúscula, por padrão.

Como exemplos, no nosso aplicativo temos a expressão EL `#{olaMB.nome}` para invocar os métodos setNome e getNome e a EL `#{olaMB.saudar}` para invocar o método saudar do Bean gerenciado OlaMB.

4.2.11 Os componentes JSF e suas funções na página xhtml

Componente	Função
`<h:head>`	Renderiza a tag HTML <head>
`<h:body>`	Renderiza a tag HTML <body>
`<h:form>`	Renderiza a tag HTML <form> e configura as propriedades method="post" e action="/JSF-Introducao/ola.xhtml". O método sempre será *post* e o *action* sempre apontará para a página que contém o formulário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

`<h:panelGrid>`	Renderiza uma Table HTML, em nosso exemplo com uma coluna `columns="1"` e 5 pixels de espaço entre o conteúdo e a borda da célula da tabela `cellpadding="5"`
`<h:outputLabel>`	Renderiza um componente alinhado como um rótulo para um componente de entrada dos dados especificados pela propriedade for="". Em nosso exemplo, para o inputText de nome nomeInput, o label foi aninhado com a propriedade `for="nomeInput"`
`<h:inputText id="nomeInput" value="#{olaMB.nome}"/>`	Renderiza a tag HTML <input type = "text"> Tem o valor ligado ao atributo nome, através dos métodos setNome e getNome da classe OlaMB. Comportamento da expressão EL `#{olaMB.nome}` • Quando a tela é carregada pela primeira vez, o método getNome será invocado e o componente será exibido com o valor vazio. • Quando o usuário digitar o nome e clicar no botão de submit, o método setNome será invocado e o atributo da classe receberá o valor digitado e submetido. • A tela será renderizada novamente, após o processamento realizado pelo post, o método getNome novamente será invocado, mas dessa vez, o atributo conterá o valor digitado, portanto, o componente será apresentado com o valor contido no atributo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

`<h:commandButton`
`id="saudarButton"`
`value="Diga olá"`
`type="submit"`
`action="#{olaMB.saudar}"`
`/>`
	Renderiza a tag HTML <input type = "submit">, um botão HTML que é capaz de submeter o formulário `type="submit"`
	
	Comportamento da expressão EL `#{olaMB.saudar}`
	
	Invoca o método saudar do Managed Bean OlaMB.

`<h:outputText`
`id="respostaOutput"`
`value="#{olaMB.resposta}"`
`/>`
	Renderiza um texto simples dentro da tag HTML .
	
	Em nosso exemplo, o texto será obtido pelo método getResposta que retorna o atributo resposta da classe OlaMB.
	
	Comportamento da expressão EL `#{olaMB.resposta}`
	
	• Quando a tela for carregada pela primeira vez, o método getResposta será invocado, como o atributo resposta estará vazio, não haverá texto para exibir.
	
	• Quando o usuário digitar o nome e clicar no botão de submit, o método saudar será invocado e o atributo resposta receberá a String "Olá!" concatenada com o atributo nome.
	
	• A tela será renderizada novamente, após o processamento realizado pelo post, o método getResposta novamente será invocado, mas dessa vez, o atributo conterá o valor concatenando, portanto, o componente será apresentado com o valor contido no atributo.

Quadro 4.1 – Executar o Projeto
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

No formulário da nossa aplicação, os componentes foram codificados com a propriedade id, preenchidas com valores que identificam unicamente cada componente do form.

4.2.12 Executar o Projeto

Execute a aplicação, botão direito em ola.xhtml, Run As > Run on Server ou use o atalho do Eclipse Alt + Shift + X, R.

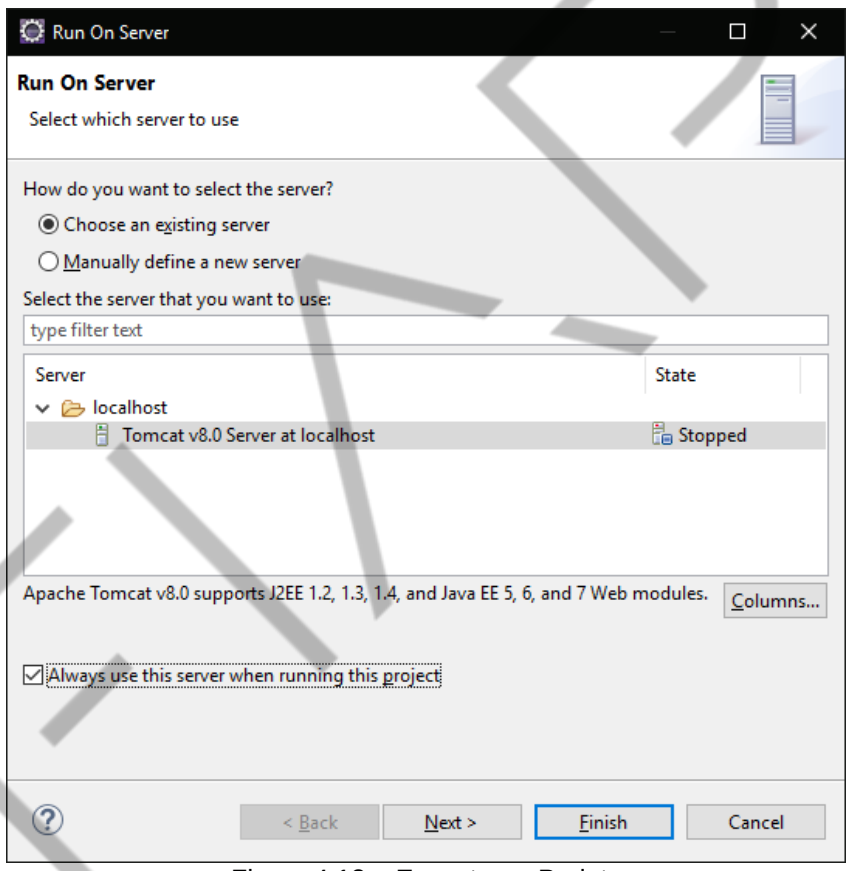

Figura 4.18 – Executar o Projeto
Fonte: Elaborado pelo autor (2018)

A tela Olá será apresentada em HTML. Como os atributos nome e resposta não possuem valor, os elementos HTML input type="text" e span não apresentarão valores.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 4.19 – Testar o Projeto(I)
Fonte: Elaborado pelo autor (2018)

Digitado o seu nome no input type="text", clique no botão Diga Olá.

Figura 4.20 – Testar o Projeto(II)
Fonte: Elaborado pelo autor (2018)

Após o processamento do post, a tela será renderizada conforme imagem abaixo. O método setNome da classe OlaMB foi invocado durante o processamento, o atributo nome recebeu o valor digitado e na sequência, o método getNome também foi invocado, portanto, o input type="text" teve sua propriedade value preenchida, com o valor do atributo nome. Esse comportamento nem sempre é desejável, geralmente, queremos que o input seja apresentado vazio.

O método saudar foi executado pelo botão Diga Olá. Podemos ver que o atributo resposta da classe foi populado com a concatenação abaixo.

```
    public void saudar() {
        System.out.println(nome);
        resposta = "Olá! " + nome;
    }
```

Código-fonte 4.7 – método saudar()
Fonte: Elaborado pelo autor (2018)

Após a execução do método saudar, o getResposta foi invocado e a tela renderizada com o texto "Olá! Fernando", exibido dentro da tag HTML span.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 4.21 – Testar o Projeto (III)
Fonte: Elaborado pelo autor (2018)

4.2.13 O código-fonte HTML retornado ao cliente

Novamente, acione o botão direito do mouse, agora na ola.xhtml e clique em Exibir código-fonte. O seguinte código HTML será apresentado. Confira as tags HTML que foram geradas pelos componentes JSF.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="j_idt2">
    <title>Seja bem vindo!</title>
</head>
<body>
<h1>Bem vindo ao mundo JSF!</h1>
<form id="saudacaoForm" name="saudacaoForm"
method="post" action="/JSF-Introducao/ola.xhtml"
enctype="application/x-www-form-urlencoded">
<input type="hidden" name="saudacaoForm"
value="saudacaoForm" />
<table id="saudacaoForm:formTable" cellpadding="5">
<tbody>
<tr>
<td><label id="saudacaoForm:nomeLabel"
for="saudacaoForm:nomeInput">Digite seu nome:</label></td>
</tr>
<tr>
<td><input id="saudacaoForm:nomeInput" type="text"
name="saudacaoForm:nomeInput" value="Fernando" /></td>
</tr>
<tr>
<td><input id="saudacaoForm:saudarButton" type="submit"
name="saudacaoForm:saudarButton" value="Diga olá" /></td>
</tr>
<tr>
<td><span id="saudacaoForm:respostaOutput">Olá!
Fernando</span></td>
</tr>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      </tbody>
      </table>
      <input    type="hidden"    name="javax.faces.ViewState"
id="j_id1:javax.faces.ViewState:0"
value="6396248710271452103:-2880765834759751 07"
autocomplete="off" />
      </form></body>
      </html>
```

Código-fonte 4.8 – código fonte HTML
Fonte: Elaborado pelo autor (2018)

4.3 Sobre o Managed Bean

Quando uma página faz referência a um *bean* gerenciado pela primeira vez, a implementação do *JavaServer Faces* o inicializa com base na anotação e no escopo da classe Bean. Note que não precisamos codificar uma linha com **new**, para instanciar a classe OlaMB em nossa aplicação de exemplo.

Um *Managed Bean*, por padrão, possui o nome da classe quando é instanciado pelo JSF e o escopo padrão. Porém, podemos definir por codificação um nome diferente e determinar quando deve ser instanciado e por quanto tempo deve viver.

Olhe o seguinte trecho.

```
@ManagedBean (name="OlaBean")
@RequestScoped
public class OlaMB implements Serializable{
```

Código-fonte 4.9 – Trecho da classe OlaMB
Fonte: Elaborado pelo autor (2018)

O código acima determina que o Managed Bean seja reconhecido como *OlaBean*, quando instanciado. Se essa alteração for aplicada em nosso exemplo, a EL utilizada na ola.xhtml deverá mudar para `#{olaBean.}`.

Sobre a anotação `@RequestScoped`, define o escopo que o Managed *Bean* será colocado, e o escopo em uma aplicação JSF determina o tempo de vida do *Bean*. Quando não utilizamos uma anotação, tal qual fizemos na OlaMB, o escopo padrão será empregado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.4 Os escopos do JavaServer Faces 2.2

Em JSF, podemos destacar os seguintes escopos:

@RequestScoped (javax.faces.bean.RequestScoped) é o padrão no JSF 2.2, há o tempo de vida enquanto durar uma resposta a uma requisição HTTP. A instância do *Managed Bean* é inicializada quando a requisição ao objeto é atendida no servidor e termina quando a resposta é enviada ao cliente.

@SessionScoped (javax.faces.bean.SessionScoped) tem a instância mantida enquanto a sessão HTTP existe, geralmente, por mais de uma requisição do cliente para a mesma página ou navegando em mais de uma página. O *bean* só é destruído quando a sessão HTTP termina. Cada cliente possui sua sessão (única) e pode interagir com o que for armazenado nela.

@ViewScoped (javax.faces.bean.ViewScoped) determina que a instância do *Managed Bean* seja mantida enquanto o cliente enviar requisições para a mesma View (página JSF) e seja destruída quando o cliente navegar para outra página. Esse escopo é muito útil quando o cliente edita componentes da mesma página, principalmente quando a página utiliza AJAX. Agora, se pretende manter dados dos componentes editados em várias páginas, visíveis apenas para o usuário que os editou, o escopo de sessão deve ser considerado.

@ApplicationScoped (javax.faces.bean.ApplicationScoped) é um escopo de aplicação e não de usuário. Portanto, mantém a instância do *Managed Bean* durante todo o tempo em que a aplicação estiver em execução. Por ter um nível de aplicação, pode ser acessado por qualquer página do aplicativo e o que for armazenado em um *bean* com esse escopo pode ser acessado, de forma compartilhada, por todos os clientes que estejam navegando na aplicação.

@FlowScoped (javax.faces.bean.FlowScoped) é basicamente o escopo que mantém a instância enquanto o cliente interage com um fluxo, um conjunto das páginas relacionadas a uma lógica dos negócios, com um ponto de entrada e um ponto de saída. Um carrinho de compras desenvolvido em várias telas pode ser considerado como um bom exemplo de fluxo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

@NoneScoped (javax.faces.bean.NoneScoped) é o escopo de vida mais curto de todos, possui como principal função servir outros *beans* gerenciados e viverá o tempo que o *bean* que o referenciou viver.

4.4.1 O atributo eager

Agora, avalie o seguinte trecho de código e considere que parte da nossa aplicação tem que atender a funcionalidade de um Chat.

```
@ManagedBean (eager=true)
@ApplicationScoped
public class ChatMB implements Serializable{
```

Código-fonte 4.10 – Trecho da classe ChatMB
Fonte: Elaborado pelo autor (2018)

Comentamos que um *bean* gerenciado é instanciado pelo JSF quando uma requisição é feita. Para uma funcionalidade de Chat, geralmente não queremos que o primeiro cliente pague o preço, esperando pela inicialização. Concorda?

Para que isso não ocorra, devemos atribuir *true* para o atributo *eager* e assim forçar que o *bean* seja inicializado no *start* da nossa aplicação. A propósito, um Chat é um bom exemplo para o emprego do escopo Application.

4.5 Bibliotecas de tags JSF

Como comentamos, a tecnologia JavaServer Faces oferece suporte a várias bibliotecas de tags para adicionar componentes a uma página Web. Em nosso exemplo, acessamos apenas uma das bibliotecas, a de componentes HTML, utilizando o prefixo `h:` e o *namespace* http://xmlns.jcp.org/jsf/html.

No desenvolvimento das páginas, podemos misturar componentes visuais e não visuais, adicionando-os por meio de tags. Como vimos, esses componentes ficam organizados em bibliotecas também conhecidas como *taglibs.*

As principais bibliotecas de tags JSF são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

URI	Prefixo	Exemplos	Funções
http://xmlns.jcp.org/jsf/html	h:	<h:form> <h:button> <h:body>	Componentes JavaServer Faces para os objetos UI.
http://xmlns.jcp.org/jsf/core	f:	<f:convertNumber> <f:convertDateTime> <f:validate> <f:ajax> <f:validateRegex>	Tags JavaServer Faces para ações adicionais, tais como, conversões, validações, ajax etc.
http://xmlns.jcp.org/jsf/facelets	ui:	<ui:insert> <ui:component>	Tags para criação de templates de páginas.
http://xmlns.jcp.org/jsf/composite	cc:	<cc:interface>	Tags para suportar composição de componentes para reusabilidade.
http://xmlns.jcp.org/jsf/passthrough	pt:	pt:type	Tags para suporte ao HTML5, como no uso de componentes para email, date, datetime etc.

Quadro 4.2 – Bibliotecas JSF
Fonte: Elaborado pelo autor (2018)

4.6 Internacionalização

A internacionalização é uma forma na qual os *labels* dos componentes da GUI, as mensagens de validação, o formato da data, entre outros não são codificados no

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

programa. São armazenados fora do código-fonte em arquivos de extensão *.properties*, que ficam em pacotes de recursos e são recuperados dinamicamente pelo *JavaServer Faces.*

O JSF fornece uma maneira muito simples de lidar com o pacote de recursos, para que as aplicações funcionem com diversos idiomas, fornecendo a possibilidade de seleção da língua em tempo de execução. Para tanto, utilizamos o Locale, que funciona como um conjunto de informações que descreve os formatos e as mensagens de uma determinada língua e/ou país.

No geral, o processo inicia quando a informação do idioma é enviada ao browser, o JSF começa o processo de busca na árvore de idiomas no faces-config.xml para identificar o recurso mais adequado e, de acordo com o que for encontrado, busca os textos em um arquivo de sentenças definido como .properties.

Na configuração baixo, o arquivo messages_pt_BR.properties ou messages_en.properties, do pacote br.com.fiapon.veiculos.resources, será utilizado, de acordo com a linguagem enviada pelo Browser.

```
<application>
        <locale-config>
              <default-locale>pt_BR</default-locale>
              <supported-locale>en</supported-locale>
        </locale-config>
        <resource-bundle>
              <base-name>
                 br.com.fiapon.veiculos.resources.messages
              </base-name>
              <var>msg</var>
        </resource-bundle>
  </application>
```

Código-fonte 4.11 – Trecho do faces-config.xml
Fonte: Elaborado pelo autor (2018)

A variável msg será empregada na página xhtml para acessarmos os textos contidos no arquivo selecionado. Veja o exemplo em um outputLabel.

```
<h:outputLabel
  for="textPass"
  value="#{msg['Solicitacao.senha']}" />
```

Código-fonte 4.12 – Trecho da página solicitacao.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.7 Navegação em JSF

Existem dois tipos de navegação no JavaServer Faces: Navegação Implícita e Navegação Explícita.

4.7.1 Navegação implícita (por links e botões)

Quando um usuário clica em um botão ou link, uma String (outcome) é enviada para o JSF e o outcome será utilizado para a navegação, definindo a próxima tela que será exibida no browser.

Nos códigos abaixo, a tela index.xhtml será chamada, mesmo sem a extensão .xhtml.

```
<h:link outcome="index">
      <h:outputText value="Home" />
</h:link>
```

```
<h:form>
      <h:commandButton  action="index" value="Home" />
</h:form>
```

Código-fonte 4.13 – Trecho da index.xhtml
Fonte: Elaborado pelo autor (2018)

4.7.2 Navegação implícita (por retorno de um método)

Nos códigos abaixo, a tela index.xhtml também será chamada, mesmo sem a extensão .xhtml, só que, dessa vez, por meio do retorno de um método no *Managed Bean.*

Código na página xhtml de origem, o atributo immediate="true" evita qualquer validação existente no form. Quando um *action* tem uma EL chamando um método do *Bean,* o retorno do método deverá ser a String (outcome) da navegação.

```
      <h:commandButton
          value="Home"
          immediate="true"
          action="#{solicitacaoBean.navegarHome}" />
```

Código-fonte 4.14 – Trecho da solicitacao.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Código no Managed Bean SolicitacaoBean.

```
public String navegarHome() {
    return "index";
}
```

Código-fonte 4.15 – Trecho da classe SolicitacaoBean
Fonte: Elaborado pelo autor (2018)

O JSF, por padrão, chama outra página do servidor, executando a navegação pelo *servlet* (Controller) e a URL da barra de endereços do browser não muda.

Para ativar o redirecionamento da página, adicione faces-redirect=true no final do nome da página. Veja o código no *Managed Bean* SolicitacaoBean alterado.

```
public String navegarHome() {
    return "index?faces-redirect=true";
}
```

Código-fonte 4.16 – NavegarHome da classe SolicitacaoBean
Fonte: Elaborado pelo autor (2018)

O redirecionamento é um processo muito mais lento que um *forward*, pois são necessárias duas requisições e não apenas uma. Importante, os objetos colocados no escopo do request original são perdidos durante o segundo request.

4.7.3 Navegação explícita

A navegação explícita é feita através das regras de navegação configuradas no arquivo faces-config.xml. Para declarar uma regra explícita, precisamos informar o caminho da tela de origem, um *outcome* (String de navegação) e o caminho da tela de destino.

```
<navigation-rule>
    <from-view-id>/login.xhtml</from-view-id>
        <navigation-case>
            <from-outcome>certo</from-outcome>
            <to-view-id>/inicio.xhtml</to-view-id>
        </navigation-case>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            <navigation-case>
                <from-outcome>errado</from-outcome>
                <to-view-id>/erro.xhtml</to-view-id>
            </navigation-case>
    </navigation-rule>
```

Código-fonte 4.17 – Exemplo da navigation rule
Fonte: Elaborado pelo autor (2018)

4.8 Praticar em JSF

Vamos criar uma aplicação para o nosso conhecimento. Nela, utilizaremos componentes de várias taglibs, internacionalização, validações e navegações. Nossa aplicação simulará uma imobiliária online muito simples, possibilitando o envio de solicitações de opções de imóveis para aluguel ou compra.

4.8.1 Criar o Projeto no Eclipse

No Eclipse, selecione File > New > Dynamic Web Project e configure-o, conforme a imagem. Clique em Modify...

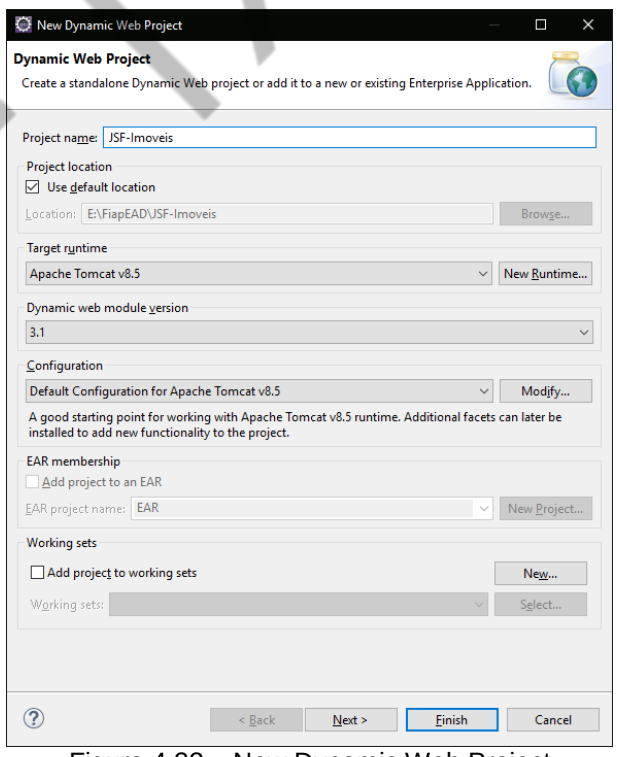

Figura 4.22 – New Dynamic Web Project
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em Project Facets, selecione Java Server faces e clique OK.

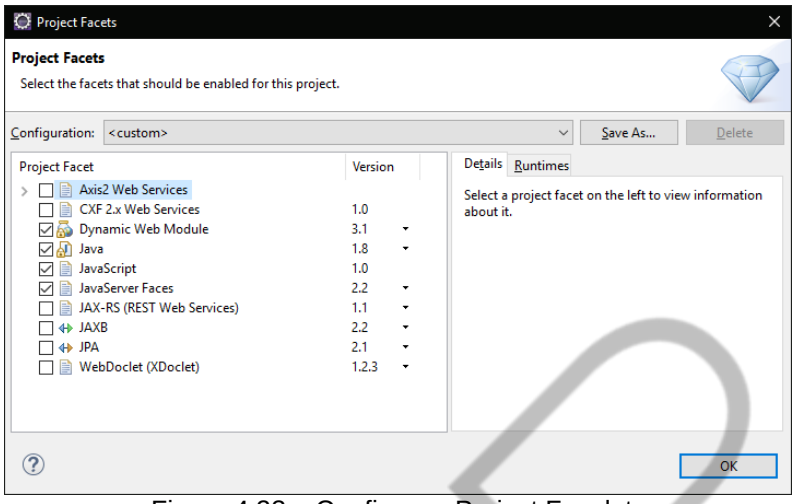

Figura 4.23 – Configurar Project Facelets
Fonte: Elaborado pelo autor (2018)

Clique em Next.

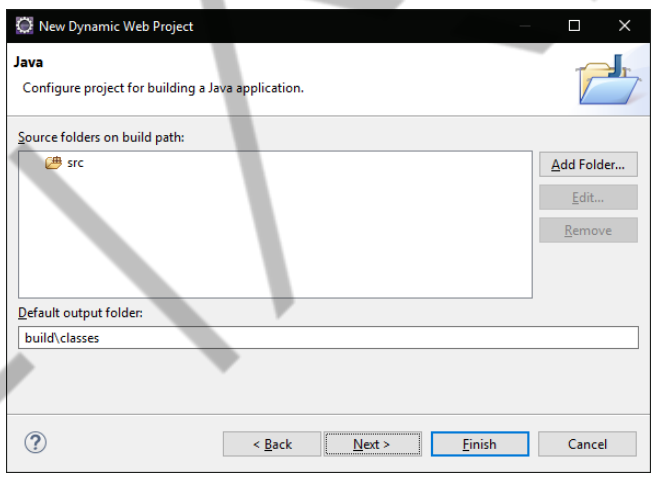

Figura 4.24 – Configurar Source Folders
Fonte: Elaborado pelo autor (2018)

Em Web Module, selecione Generate web.xml.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

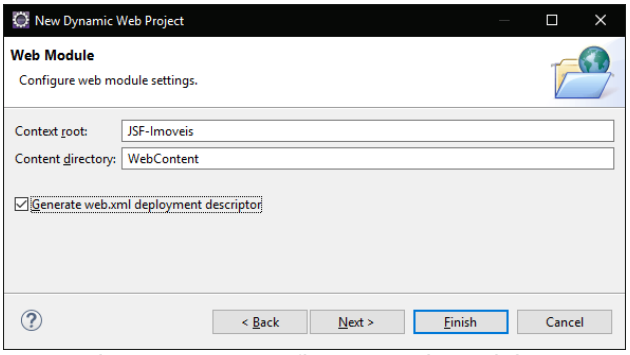

Figura 4.25 – Configurar Web Modules
Fonte: Elaborado pelo autor (2018)

Em JSF Capabilities, configure o *wizard* conforme a imagem abaixo.

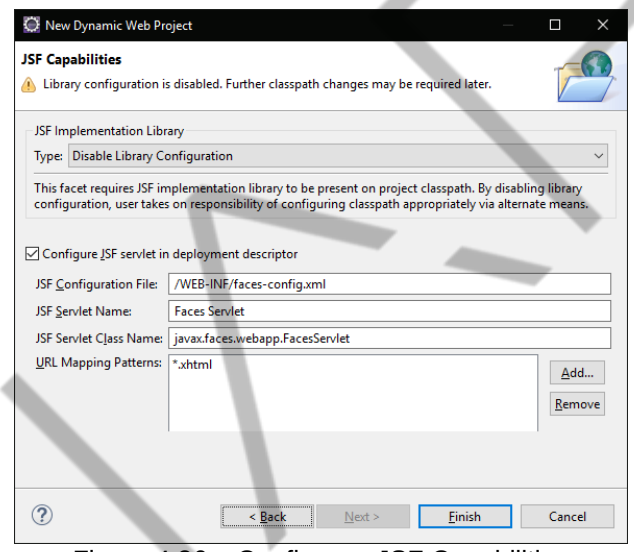

Figura 4.26 – Configurar JSF Capabilities
Fonte: Elaborado pelo autor (2018)

Agora, vamos configurar o nosso projeto do Eclipse para utilizar o *Maven*. Com o botão direito sobre o projeto, selecione Configure e converta-o para *Maven Project*, como fizemos no projeto anterior. Crie o *Maven POM*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

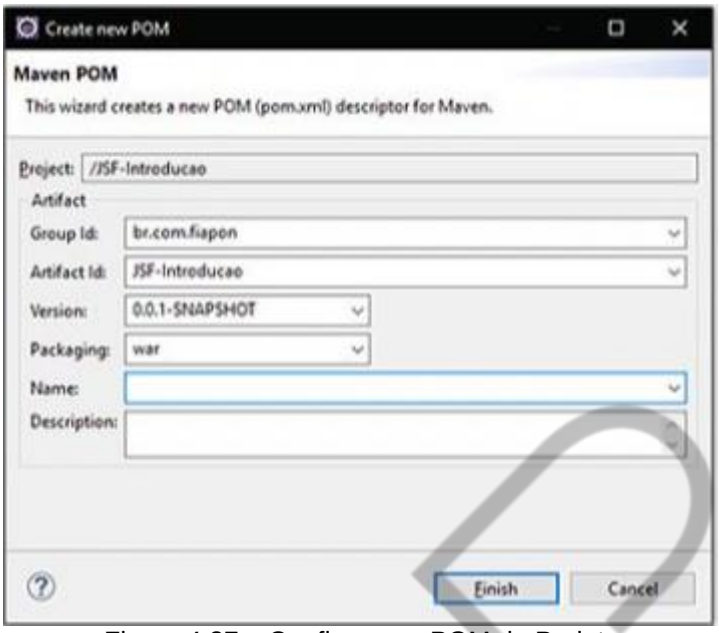

Figura 4.27 – Configurar o POM do Projeto
Fonte: Elaborado pelo autor (2018)

Inclua a dependência abaixo de *war*, para facilitar a nossa vida com o Tomcat, utilizaremos a versão 2.2.2 do Mojarra.

```
<dependencies>
    <!-- Mojarra 2.2.2 -->
    <dependency>
        <groupId>org.glassfish</groupId>
        <artifactId>javax.faces</artifactId>
        <version>2.2.2</version>
        <scope>compile</scope>
    </dependency>
</dependencies>
```

Código-fonte 4.18 – POM do Projeto
Fonte: Elaborado pelo autor (2018)

4.8.2 Criar o back-end da aplicação

Na pasta src, crie os seguintes pacotes e arquivos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

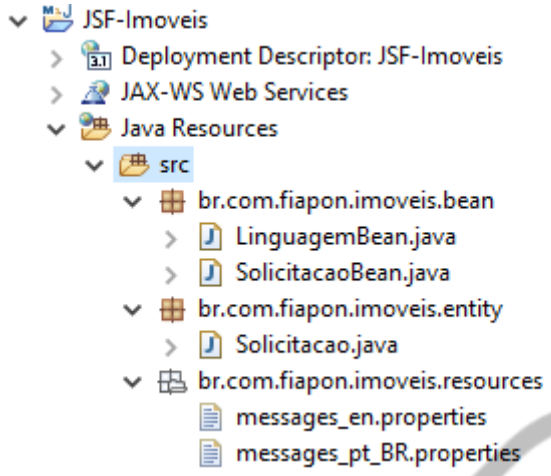

Figura 4.28 – A Estrutura do Projeto
Fonte: Elaborado pelo autor (2018)

4.8.2.1 A classe entity Solicitacao.java

A classe Solicitacao será instanciada pelo *Managed Bean* SolicitacaoBean e receberá os valores digitados pelo usuário na página xhtml de solicitação de um imóvel para aluguel ou compra.

```
package br.com.fiapon.imoveis.entity;

import java.io.Serializable;
import java.util.Calendar;
import java.util.List;

public class Solicitacao implements Serializable {

    private static final long serialVersionUID = 1L;
    private int codigo;
    private String cidade;
    private float valor;
    private String descricao;
    private String senha;
    private boolean novo;
    private List<String> itens;
    private List<String> comodos;
    private String tipo;
    private String finalidade;
    private String operacao;
    private String email;
    private Calendar dataSolicitacao;

    public Solicitacao() {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        //Inicializa a data de solicitação
        dataSolicitacao= Calendar.getInstance();
    }

    public int getCodigo() {
        return codigo;
    }

    public String getTipo() {
        return tipo;
    }

    public void setTipo(String tipo) {
        this.tipo = tipo;
    }

    public String getFinalidade() {
        return finalidade;
    }

    public void setFinalidade(String finalidade) {
        this.finalidade = finalidade;
    }

    public String getOperacao() {
        return operacao;
    }

    public void setOperacao(String operacao) {
        this.operacao = operacao;
    }

    public void setCodigo(int codigo) {
        this.codigo = codigo;
    }

    public String getCidade() {
        return cidade;
    }

    public void setCidade(String cidade) {
        this.cidade = cidade;
    }

    public float getValor() {
        return valor;
    }

    public void setValor(float valor) {
        this.valor = valor;
    }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    public String getDescricao() {
        return descricao;
    }

    public void setDescricao(String descricao) {
        this.descricao = descricao;
    }

    public String getSenha() {
        return senha;
    }

    public void setSenha(String senha) {
        this.senha = senha;
    }

    public boolean isNovo() {
        return novo;
    }

    public void setNovo(boolean novo) {
        this.novo = novo;
    }

    public List<String> getItens() {
        return itens;
    }

    public void setItens(List<String> itens) {
        this.itens = itens;
    }

    public List<String> getComodos() {
        return comodos;
    }

    public void setComodos(List<String> comodos) {
        this.comodos = comodos;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public Calendar getDataSolicitacao() {
        return dataSolicitacao;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        }

        public void setDataSolicitacao(Calendar dataSolicitacao)
{
            this.dataSolicitacao = dataSolicitacao;
        }

    }
```

Código-fonte 4.19 – Classe Solicitacao.java
Fonte: Elaborado pelo autor (2018)

4.8.2.2 A classe Managed Bean SolicitacaoBean.java

A classe *SolicitacaoBean* será responsável por instanciar uma solicitação, simular o cadastro, navegar para a página inicial e retornar Listas para popular a tela, tratando internacionalização. Alguns códigos são novos, as explicações estão nos comentários.

A anotação @PostConstruct garante que o método init() será disparado quando o *bean* for totalmente inicializado.

```
package br.com.fiapon.imoveis.bean;

import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

import javax.annotation.PostConstruct;
import javax.faces.application.FacesMessage;
import javax.faces.bean.ManagedBean;
import javax.faces.context.FacesContext;

import br.com.fiapon.imoveis.entity.Solicitacao;

@ManagedBean
public class SolicitacaoBean {

    private Solicitacao solicitacao;
    private Locale locale;

    @PostConstruct // Inicializando os atributos
    private void init() {
        solicitacao = new Solicitacao();
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     }

     public void cadastrar() {
          System.out.println("Cidade " +
solicitacao.getCidade());
          System.out.println("Valor: " +
solicitacao.getValor());
          System.out.println("Desc: " +
solicitacao.getDescricao());
          System.out.println("Senha: " +
solicitacao.getSenha());
          System.out.println("Novo: " +
solicitacao.isNovo());
          System.out.println(solicitacao.getItens());
          System.out.println(solicitacao.getComodos());
          System.out.println(solicitacao.getTipo());
          System.out.println(solicitacao.getFinalidade());
          System.out.println(solicitacao.getOperacao());
          System.out.println(solicitacao.getEmail());

     // Mensagem que será associada ao componente
h:h:messages - Por ser um simulação, não está
internacionalizado
          FacesMessage msg = new FacesMessage("Imóvel
Cadastrado!");


          //Adiciona a menssagem na instância corrente da
solicitação
          FacesContext.getCurrentInstance().addMessage(null,
msg);

     }

    //Navega para a tela home

     public String navegarHome() {
          return "index?faces-redirect=true";
     }

     //Retorna true se a linguagem for pt
     //Novamente utilizamos o contexto, agora para obter o
Locale
     private boolean getLocalePt() {
          locale =
FacesContext.getCurrentInstance().getViewRoot().getLocale();

          return
locale.getLanguage().toString().equals("pt");
     }
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    //Retorna uma lista de tipos para a página xhtml, tratando
internacionalização

    public List<String> getTipos() {
        List<String> lista = new ArrayList<String>();

        if (getLocalePt()) {
            lista.add("Casa");
            lista.add("Apartamento");
            lista.add("Armazém");
            lista.add("Sitio");
            lista.add("Fazenda");
        } else {
            lista.add("House");
            lista.add("Apartment");
            lista.add("Warehouse");
            lista.add("Rural");
            lista.add("Farm");
        }

        return lista;
    }

    //Retorna uma lista de comodos para a página xhtml,
tratando internacionalização

    public List<String> getListaComodos() {
        List<String> lista = new ArrayList<String>();

        if (getLocalePt()) {
            lista.add("Quarto");
            lista.add("Sala");
            lista.add("Cozinha");
            lista.add("Banheiro");
            lista.add("Varanda");
        } else {
            lista.add("Bedroom");
            lista.add("Living room");
            lista.add("Kitchen");
            lista.add("Bathroom");
            lista.add("Balcony");
        }
        return lista;
    }

    public Solicitacao getSolicitacao() {
        return solicitacao;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
	}

	public void setSolicitacao(Solicitacao solicitacao) {
		this.solicitacao = solicitacao;
	}

}
```

Código-fonte 4.20 – Classe SolicitacaoBean.java
Fonte: Elaborado pelo autor (2018)

4.8.2.3 A classe Managed Bean LinguangemBean.java

A classe LinguagemBean será responsável por trocar o *Locale* para inglês ou português, de acordo com a escolha do usuário. É um *bean* gerenciado com escopo de sessão, ou seja, após a linguagem trocada, o usuário poderá navegar para outra página e o arquivo das propriedades correspondente será utilizado pelo JSF.

```
package br.com.fiapon.imoveis.bean;

import java.io.Serializable;
import java.util.Locale;

import javax.annotation.PostConstruct;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.SessionScoped;
import javax.faces.component.UIViewRoot;
import javax.faces.context.FacesContext;

@ManagedBean
@SessionScoped
public class LinguagemBean implements Serializable {

	private static final long serialVersionUID = 1L;

	private Locale locale;

	@PostConstruct
	private void init(){
		locale =
FacesContext.getCurrentInstance().getViewRoot()
			.getLocale();
	}

	public void mudarParaIngles() {
		FacesContext context =
FacesContext.getCurrentInstance();
		UIViewRoot view = context.getViewRoot();
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            locale = Locale.ENGLISH;
            view.setLocale(locale);
      }

      public void mudarParaPortugues() {

            FacesContext context =
FacesContext.getCurrentInstance();
            UIViewRoot view = context.getViewRoot();
            locale = new Locale("pt", "BR");
            view.setLocale(locale);
      }

      public Locale getLocale() {
            return locale;
      }

      public void setLocale(Locale locale) {
            this.locale = locale;
      }
}
```

Código-fonte 4.21 – Classe LinguagemBean.java
Fonte: Elaborado pelo autor (2018)

4.8.2.4 O arquivo de propriedades messages_en.properties

O JSF obterá textos para rótulos, valores e mensagens desse arquivo, se a linguagem enviada pelo browser for inglês - en. Nossa aplicação terá as páginas apenas index.xhtml e solicitacao.xhtml internacionalizadas.

```
# -- Index --
Index.titulo=FIAPON PROPERTIES
Index.saudacao=Welcome\!
Index.mensagem= JSF system to rent and buy properties
Index.solicitacao.mensagem=Request Properties
Index.ouMensagem=\ or
Index.consultar.mensagem=Search for properties

# -- Solicitacao --
Solicitacao.titulo=Request properties
Solicitacao.cidade=City
Solicitacao.cidadeRequerida= City is required
Solicitacao.cidadeInvalida= City invalid content


Solicitacao.tipo=Propertie types
Solicitacao.tipoRequerido= Property type is required
Solicitacao.tipoSelecione= Select
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Solicitacao.operacao= Goal
Solicitacao.operacaoRequerida= The goal is required
Solicitacao.operacao.aluguel= Rent
Solicitacao.operacao.compra= Buy

Solicitacao.valor= Approximate value
Solicitacao.valorInvalido= Value must be a number consisting
of one or more digits

Solicitacao.finalidade= For
Solicitacao.finalidadeRequerida= The field For is required
Solicitacao.finalidade.comercial= Commercial
Solicitacao.finalidade.residencial= Residential

Solicitacao.descricao=Description
Solicitacao.novo=New

Solicitacao.itens=Items
Solicitacao.itens.piscina=Swimming pool
Solicitacao.itens.hidromassagem=Whirlpool / Hot tub
Solicitacao.itens.academia=Gym academy
Solicitacao.itens.sauna=Sauna

Solicitacao.comodos=Rooms
Solicitacao.senha=Password

Solicitacao.email=E-mail
Solicitacao.emailRequerido=The e-mail is required
Solicitacao.emailInvalido=Enter your valid email address

Solicitacao.data=Date
Solicitacao.dataFormato=MM/dd/yyyy
```

Código-fonte 4.22 – Messages_en.properties
Fonte: Elaborado pelo autor (2018)

4.8.2.5 O arquivo de propriedades messages_pt_BR.properties

O JSF obterá textos para rótulos, valores e mensagens desse arquivo, se a linguagem enviada pelo browser for português - pt_BR. Nossa aplicação terá as páginas apenas index.xhtml e solicitacao.xhtml internacionalizadas. Claro, as parametrizações são as mesmas, só que em português.

```
# -- Index --
Index.titulo=FIAPON IMÓVEIS
Index.saudacao=Bem vindo\!
Index.mensagem=Sistema JSF para aluguel e compra de imóveis.
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Index.solicitacao.mensagem=Solicitar Imóvel
Index.ouMensagem=\ ou
Index.consultar.mensagem=Consultar Imóveis

# -- Solicitacao --
Solicitacao.titulo=Solicitação de Imóvel
Solicitacao.cidade=Cidade
Solicitacao.cidadeRequerida= A cidade é obrigatória!
Solicitacao.cidadeInvalida= Nome de cidade inválido

Solicitacao.tipo=Tipo
Solicitacao.tipoRequerido= O tipo é requerido!
Solicitacao.tipoSelecione= Selecione

Solicitacao.operacao= Objetivo
Solicitacao.operacaoRequerida= O objetivo é requerido!
Solicitacao.operacao.aluguel= Aluguel
Solicitacao.operacao.compra= Compra

Solicitacao.valor= Valor aproximado
Solicitacao.valorInvalido= Valor deve ser numérico

Solicitacao.finalidade= Finalidade
Solicitacao.finalidadeRequerida= A finalidade é requerida!
Solicitacao.finalidade.comercial= Comercial
Solicitacao.finalidade.residencial= Residencial

Solicitacao.descricao=Descrição
Solicitacao.novo=Novo

Solicitacao.itens=Itens
Solicitacao.itens.piscina=Piscina
Solicitacao.itens.hidromassagem=Hidromassagem
Solicitacao.itens.academia=Academia
Solicitacao.itens.sauna=Sauna

Solicitacao.comodos=Cômodos
Solicitacao.senha=Senha

Solicitacao.email=E-mail
Solicitacao.emailRequerido=O e-mail é requerido
Solicitacao.emailInvalido=Digite um e-mail válido

Solicitacao.data=Data
Solicitacao.dataFormato=dd/MM/yyyy
```

Código-fonte 4.23 – Messages_pt_BR.properties
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.8.2.6 Configurar o faces-config.xml da aplicação

Para que o JSF obtenha textos dos arquivos de propriedade da aplicação, precisamos configurar a linguagem padrão (pt_BR) a linguagem ou linguagens suportadas (en), onde estão os arquivos e a variável (msg) que será utilizada para acessar os valores nas páginas xhtml. Para funcionar, o nome dos arquivos deve respeitar o formato messages_<locale>.properties. Onde <locale> pode ser pt_BR e em, em nossa aplicação.

```
<?xml version="1.0" encoding="UTF-8"?>
<faces-config
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
    http://xmlns.jcp.org/xml/ns/javaee/web-
facesconfig_2_2.xsd"
    version="2.2">

    <application>
          <locale-config>
               <default-locale>pt_BR</default-locale>
               <supported-locale>en</supported-locale>
          </locale-config>
          <resource-bundle>
               <base-
name>br.com.fiapon.imoveis.resources.messages</base-name>
               <var>msg</var>
          </resource-bundle>
     </application>
</faces-config>
```

Código-fonte 4.24 – Faces-config.xml
Fonte: Elaborado pelo autor (2018).

4.8.3 Criar o front-end da aplicação

4.8.3.1 Os arquivos de recursos da aplicação (img e css)

Na pasta WebContent, crie os seguintes pacotes e arquivo css. Copie e cole as imagens dentro da pasta img.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

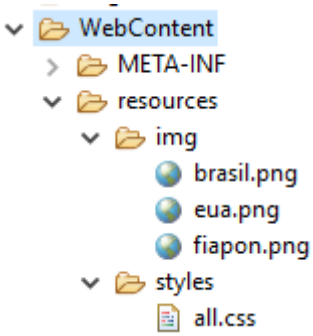

Figura 4.29 – Recursos do Projeto
Fonte: Elaborado pelo autor (2018)

4.8.3.2 O arquivo css

A nossa aplicação utilizará um template (_template), por meio da biblioteca *Facelets*, portanto, vamos criar estilos para o cabeçalho das páginas (header), para o conteúdo (content) e para o rodapé (footer).

```
@CHARSET "UTF-8";

* {
     margin: 0;
}

html, body {
     font: 1em;
     font-family: Arial, Helvetica, sans-serif;
     height: 100%;
}

#header {
     background: black;
     width: 100%;
     min-height: 100%;
}

#content {
     width: 800px;
     background-color: white;
     border-radius: 3px;
     display: flex;
     flex-direction: row;
     flex-wrap: wrap;
     justify-content: center;
     align-items: center;
     margin: auto;
     padding: 3px;
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
	padding-bottom: 12px;
}

#footer {
	background-color: black;
	width: 100%;
	font-size: 0.7em;
	color: white;
	height: 12px;
	margin-top: -12px;
}

#languages {
	position: absolute;
	top: 0;
	right: 0;
}

.submit {
	width: auto;
	padding: 0.3em 1.2em;
	background: black;
	border: 0;
	font-size: 1em;
	color: white;
	border-radius: 5px;
}

.infoMsg {
	color: green;
}

.erroMsg {
	color: red;
	padding-left: 0.5em;
	}
```

Código-fonte 4.25 – all.css
Fonte: Elaborado pelo autor (2018)

4.8.3.3 Os templates da aplicação

Na pasta WebContent, crie o pacote templates e arquivos xhtml – HTML5.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

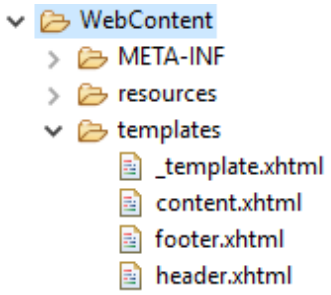

Figura 4.30 – Os Templates do Projeto
Fonte: Elaborado pelo autor (2018)

4.8.3.3.1 A página _template.xtml

Esta página utiliza a taglib *Facelets* para estruturar e inserir as páginas header, content e footer nessa ordem, bem como referenciar o arquivo all.css.

Perceba que o content fica dentro da *div header*, para que o footer não sobreponha nenhum deles. Veja como eles se acertam no css (#content e #footer).

A tag `<f:view locale="#{linguagemBean.locale}"/>` manterá a linguagem definida na sessão pelo *Managed Bean LinguagemBean* nas páginas que formam inseridas no template.

```
<!DOCTYPE html >

<html xmlns="http://www.w3.org/1999/xhtml"
     xmlns:h="http://xmlns.jcp.org/jsf/html"
     xmlns:f="http://xmlns.jcp.org/jsf/core"
     xmlns:ui="http://xmlns.jcp.org/jsf/facelets">

<f:view locale="#{linguagemBean.locale}"/>
<h:head>  <title>
          <ui:insert name="title">
               <h:outputText value="#{msg['Index.title']}" />
          </ui:insert>
     </title>
     <h:outputStylesheet
            name="all.css" library="styles" target="head" />
</h:head>

<h:body>


     <div id="header">
          <ui:insert name="header">
               <ui:include src="header.xhtml" />
          </ui:insert>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            <div id="content">
                <ui:insert name="content">
                    <ui:include src="content.xhtml" />
                </ui:insert>
            </div>
        </div>

        <div id="footer">
            <ui:insert name="footer">
                <ui:include src="footer.xhtml" />
            </ui:insert>
        </div>

</h:body>

</html>
```

Código-fonte 4.26 – _template.xhtml
Fonte: Elaborado pelo autor (2018)

4.8.3.3.2 A página header.xtml

Esta página utiliza a taglib *Facelets* para fazer uma composição de cabeçalho, incluindo uma imagem da FIAP como link para a página index.xhtml e duas imagens de bandeiras, para que os commandLinks chamem os métodos do Managed Bean que trocam o Locale.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="http://xmlns.jcp.org/jsf/html"
      xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
<h:body>
      <ui:composition>
            <div id="header">
                  <h:outputLink styleClass="headerlink"
value="index.xhtml">
                        <h:graphicImage
url="resources/img/fiapon.png" />
                  </h:outputLink>
                  <h:form>
                        <div id="languages">
                              <h:commandLink
action="#{linguagemBean.mudarParaPortugues()}"
                                    ajax="false">
                                    <h:graphicImage
name="brasil.png" library="img" />
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                            </h:commandLink>
                            <h:commandLink
action="#{linguagemBean.mudarParaIngles()}"
                                 ajax="false">
                                 <h:graphicImage name="eua.png"
library="img" />
                            </h:commandLink>
                        </div>
                    </h:form>
            </div>
        </ui:composition>
</h:body>
</html>
```

Código-fonte 4.27 – header.xhtml
Fonte: Elaborado pelo autor (2018)

4.8.3.3.3 A página content.xtml

Esta é uma página padrão que terá o conteúdo trocado pelas páginas index.xhtml, solicitacao.xhtml e construcao.xhtml.

```
<!DOCTYPE >
<html      xmlns="http://www.w3.org/1999/xhtml"
     xmlns:h="http://xmlns.jcp.org/jsf/html"
     xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
    <h:body>
        <ui:composition>
            <div id="divContent">
                <h:outputText value="#{msg['Index.saudacao']}"
/>
            </div>
        </ui:composition>
    </h:body>
</html>
```

Código-fonte 4.28 – context.xhtml
Fonte: Elaborado pelo autor (2018)

4.8.3.3.4 A página footer.xtml

Esta é uma página padrão que apresentará um rodapé com o Copyright.

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
     xmlns:h="http://xmlns.jcp.org/jsf/html"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      xmlns:ui="http://xmlns.jcp.org/jsf/facelets">
      <h:body>
           <ui:composition>
                <div>
                     Copyright © FIAPON. 2018
                </div>
           </ui:composition>
      </h:body>
</html>
```

Código-fonte 4.29 – footer.xhtml
Fonte: Elaborado pelo autor (2018)

4.8.3.4 As páginas que atendem ao negócio

Ainda na pasta WebContent, crie as seguintes páginas xhtml – HTML5.

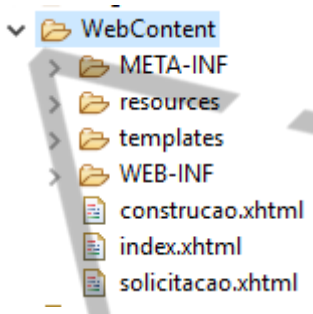

Figura 4.31 – Páginas de Negócio do Projeto
Fonte: Elaborado pelo autor (2018)

4.8.3.4.1 A página construcao.xhtml

Esta é uma página muito simples (sem internacionalização) que será chamada quando o conteúdo estiver em construção. A tag `<ui:define name="content">` é a responsável por prover o layout que substituirá o conteúdo da div content, do arquivo _template.xhtml.

```
<!DOCTYPE html>
<ui:composition template="/templates/_template.xhtml"
      xmlns="http://www.w3.org/1999/xhtml"
      xmlns:f="http://xmlns.jcp.org/jsf/core"
      xmlns:h="http://xmlns.jcp.org/jsf/html"
      xmlns:ui="http://xmlns.jcp.org/jsf/facelets">

      <ui:define name="title">
           <h:outputText value="#{msg['Index.titulo']}" />
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    </ui:define>

    <ui:define name="content">
        <div id="divContent">
            <h3>
                <h:outputText value="Em construção!" />
            </h3>
            <h:link outcome="index">
                <h:outputText value="Home" />
            </h:link>
        </div>
    </ui:define>
</ui:composition>
```

Código-fonte 4.30 – construcao.xhtml
Fonte: Elaborado pelo autor (2018)

4.8.3.4.2 A página index.xhtml

É uma página muito simples que será chamada ao iniciar a aplicação. Garanta que o web.xml possui a seguinte parametrização, abaixo de `<display-name>`.

```
  <welcome-file-list>
    <welcome-file>index.xhtml</welcome-file>
  </welcome-file-list>
```

Código-fonte 4.31 – trecho web.xml
Fonte: Elaborado pelo autor (2018)

Apesar de simples, esta página utiliza internacionalização. Perceba que a EL *#{msg['']}"* é utilizada para acessar a var msg, configurada no faces-config.xml e, por sua vez, os textos parametrizados nos arquivos das propriedades. Os textos serão buscados nos arquivos, de acordo com o *Locale*.

```
<!DOCTYPE html>
<ui:composition template="/templates/_template.xhtml"
    xmlns="http://www.w3.org/1999/xhtml"
    xmlns:f="http://xmlns.jcp.org/jsf/core"
    xmlns:h="http://xmlns.jcp.org/jsf/html"
    xmlns:ui="http://xmlns.jcp.org/jsf/facelets">

    <ui:define name="title">
        <h:outputText value="#{msg['Index.titulo']}" />
    </ui:define>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     <ui:define name="content">
          <div id="divContent">
               <h3>
                    <h:outputText
value="#{msg['Index.saudacao']}" />
               </h3>
               <p>
                    <h:outputText
value="#{msg['Index.mensagem']}" />
               </p>
               <h:link outcome="solicitacao">
                    <h:outputText
value="#{msg['Index.solicitacao.mensagem']}" />
               </h:link>
               <h:outputText
value="#{msg['Index.ouMensagem']}" />
               <h:link outcome="construcao">
                    <h:outputText
value="#{msg['Index.consultar.mensagem']}" />
               </h:link>

          </div>
     </ui:define>
</ui:composition>
```

Código-fonte 4.32 – index.xhtml
Fonte: Elaborado pelo autor (2018)

4.8.3.4.3 A página solicitacao.xhtml

A página mais complexa, além de utilizar internacionalização, possui validações e apresenta as mensagens em componentes h:messages (usado para exibir todas as mensagens armazenadas no contexto, durante o ciclo de vida do JSF) e h:message (usado para exibir uma única mensagem para um componente específico). Ou seja, o que for exibido em message, também será em messages por padrão.

Como comentamos, nesta página veremos validações que controlam se um conteúdo foi digitado ou selecionado (`required="true"`) e que configuram a mensagem da validação (`requiredMessage`). Também encontramos um exemplo de validação utilizando Expressão Regular, nesse caso, `<f:validateRegex>` para validar e-mail.

```
<f:validateRegex
     pattern="^[_A-Za-z0-9-\+]+(\.[_A-Za-z0-9-
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      ]+)*@[A-Za-z0-9-]+(\.[A-Za-z0-9]+)*(\.[A-Za-

      z]{2,})$" />
```

Código-fonte 4.33 – Exemplo Regex
Fonte: Elaborado pelo autor (2018)

O código da página solicitacao.xhtml.

```
<!DOCTYPE html>
<ui:composition template="/templates/_template.xhtml"
     xmlns="http://www.w3.org/1999/xhtml"
     xmlns:f="http://xmlns.jcp.org/jsf/core"
     xmlns:h="http://xmlns.jcp.org/jsf/html"
     xmlns:ui="http://xmlns.jcp.org/jsf/facelets"
     xmlns:pt="http://xmlns.jcp.org/jsf/passthrough">

     <ui:define name="title">
          <h:outputText value="#{msg['Index.titulo']}" />
     </ui:define>

     <ui:define name="content">
          <div>
               <h1>#{msg['Solicitacao.titulo']}</h1>
               <h:form>
                    <h:messages id="msgErro"
styleClass="erroMsg" />

                    <h:panelGrid columns="2" cellpadding="2">

                         <h:outputLabel
value="#{msg['Solicitacao.cidade']}"
                              for="textCidade" />
                         <h:panelGroup layout="block">
                              <h:inputText id="textCidade"
size="30" alt="Cidade"

     value="#{solicitacaoBean.solicitacao.cidade}"
                                   autocomplete="off"
                                   required="true"

     requiredMessage="#{msg['Solicitacao.cidadeRequerida']}"
/>
                              <h:message for="textCidade"
styleClass="erroMsg" />
                         </h:panelGroup>

                         <h:outputLabel
value="#{msg['Solicitacao.tipo']}" for="selectTipo" />
                         <h:selectOneMenu id="selectTipo"
value="#{solicitacaoBean.solicitacao.tipo}"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                required="true"

     requiredMessage="#{msg['Solicitacao.tipoRequerido']}">
                                <f:selectItem itemValue=""
itemLabel="#{msg['Solicitacao.tipoSelecione']}" />
                                <f:selectItems
value="#{solicitacaoBean.tipos}" />
                           </h:selectOneMenu>

                           <h:outputLabel
value="#{msg['Solicitacao.operacao']}"
                                for="oneRadioOp" />
                           <h:selectOneRadio id="oneRadioOp"

     value="#{solicitacaoBean.solicitacao.operacao}"
required="true"

     requiredMessage="#{msg['Solicitacao.operacaoRequerida']}
">
                                <f:selectItem
itemLabel="#{msg['Solicitacao.operacao.aluguel']}"
itemValue="Aluguel" />
                                <f:selectItem
itemLabel="#{msg['Solicitacao.operacao.compra']}"
itemValue="Compra" />
                           </h:selectOneRadio>

                           <h:outputLabel
value="#{msg['Solicitacao.finalidade']}"
                                for="oneListboxFinalidade" />
                           <h:selectOneListbox
id="oneListboxFinalidade"

     value="#{solicitacaoBean.solicitacao.finalidade}"
required="true"

     requiredMessage="#{msg['Solicitacao.finalidadeRequerida'
]}">
                                <f:selectItem
itemLabel="#{msg['Solicitacao.finalidade.comercial']}"
itemValue="Comercial" />
                                <f:selectItem
itemLabel="#{msg['Solicitacao.finalidade.residencial']}"
itemValue="Residencial" />
                           </h:selectOneListbox>

                           <h:outputLabel
value="#{msg['Solicitacao.valor']}" for="textValor" />
                           <h:inputText id="textValor"
size="30"
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     value="#{solicitacaoBean.solicitacao.valor}"

     validatorMessage="#{msg['Solicitacao.valorInvalido']}"/>

                         <h:outputLabel for="textareaDesc"
value="#{msg['Solicitacao.descricao']}" />
                         <h:inputTextarea id="textareaDesc"
rows="5" cols="40"

     value="#{solicitacaoBean.solicitacao.descricao}" />

                         <h:outputLabel
value="#{msg['Solicitacao.novo']}" for="checkboxNovo" />
                         <h:selectBooleanCheckbox
id="checkboxNovo"

     value="#{solicitacaoBean.solicitacao.novo}" />

                         <h:outputLabel
value="#{msg['Solicitacao.itens']}" for="manyCheckboxItens"
/>
                         <h:selectManyCheckbox
id="manyCheckboxItens"
value="#{solicitacaoBean.solicitacao.itens}">
                             <f:selectItem
itemLabel="#{msg['Solicitacao.itens.piscina']}"
itemValue="Piscina" />
                             <f:selectItem
itemLabel="#{msg['Solicitacao.itens.hidromassagem']}"
itemValue="Hidro" />
                             <f:selectItem
itemLabel="#{msg['Solicitacao.itens.academia']}"
itemValue="Academia" />
                             <f:selectItem
itemLabel="#{msg['Solicitacao.itens.sauna']}"
itemValue="Sauna" />
                         </h:selectManyCheckbox>


                         <h:outputLabel
value="#{msg['Solicitacao.comodos']}"
for="manyListboxComodos" />
                         <h:selectManyListbox
id="manyListboxComodos"

     value="#{solicitacaoBean.solicitacao.comodos}">
                             <f:selectItems
value="#{solicitacaoBean.listaComodos}" />
                         </h:selectManyListbox>
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                        <h:outputLabel for="textEmail"
value="#{msg['Solicitacao.email']}" />
                        <h:inputText id="textEmail"
                             size="30"

     value="#{solicitacaoBean.solicitacao.email}"
                             required="true"

     requiredMessage="#{msg['Solicitacao.emailRequerido']}"

     validatorMessage="#{msg['Solicitacao.emailInvalido']}" >
                             <f:validateRegex
                                  pattern="^[_A-Za-z0-9-
\+]+(\.[_A-Za-z0-9-]+)*@[A-Za-z0-9-]+(\.[A-Za-z0-9]+)*(\.[A-
Za-z]{2,})$" />
                        </h:inputText>

                        <h:outputLabel for="textPass"
value="#{msg['Solicitacao.senha']}" />
                        <h:inputSecret id="textPass"
                             size="30"

     value="#{solicitacaoBean.solicitacao.senha}" />

                        <h:outputLabel
value="#{msg['Solicitacao.data']}" for="textData" />
                        <h:inputText id="textData"
readonly="true" size="30"

     value="#{solicitacaoBean.solicitacao.dataSolicitacao.tim
e}">
                             <f:convertDateTime
pattern="#{msg['Solicitacao.dataFormato']}" />
                        </h:inputText>

                        <h:commandButton styleClass="submit"
value="Enviar"

     actionListener="#{solicitacaoBean.cadastrar}" />

                        <h:commandButton styleClass="submit"
value="Home" immediate="true"

     action="#{solicitacaoBean.navegarHome}" />
                    </h:panelGrid>
                </h:form>
            </div>
     </ui:define>
</ui:composition>
```

Código-fonte 4.34 – solicitação.xhtml
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.8.4 Executar o Projeto

Execute a aplicação, botão direito em index.xhtml, Run As > Run on Server ou use o atalho do Eclipse Alt + Shift + X, R.

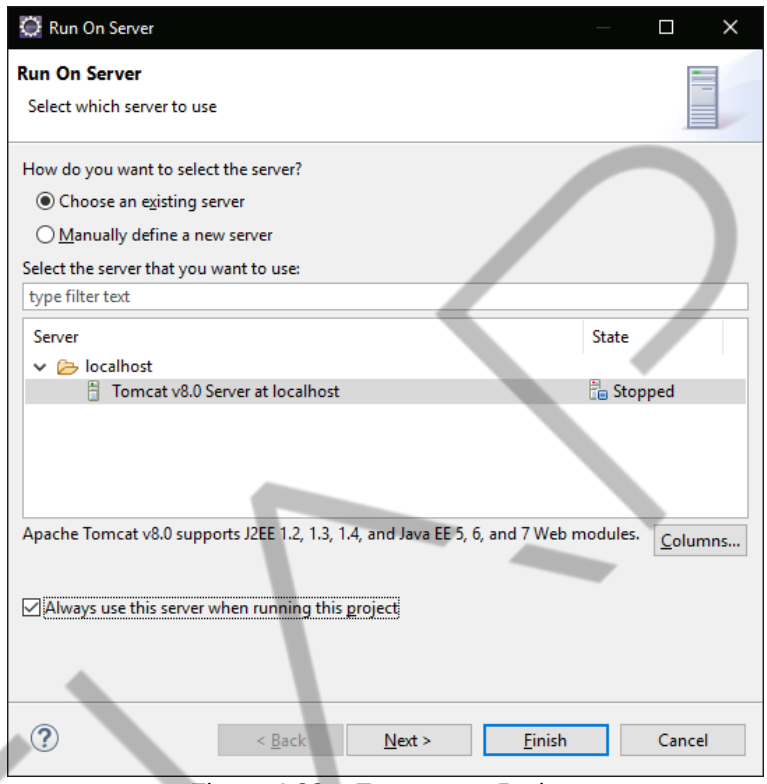

Figura 4.32 – Executar o Projeto
Fonte: Elaborado pelo autor (2018)

A tela index.xhtml será exibida.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

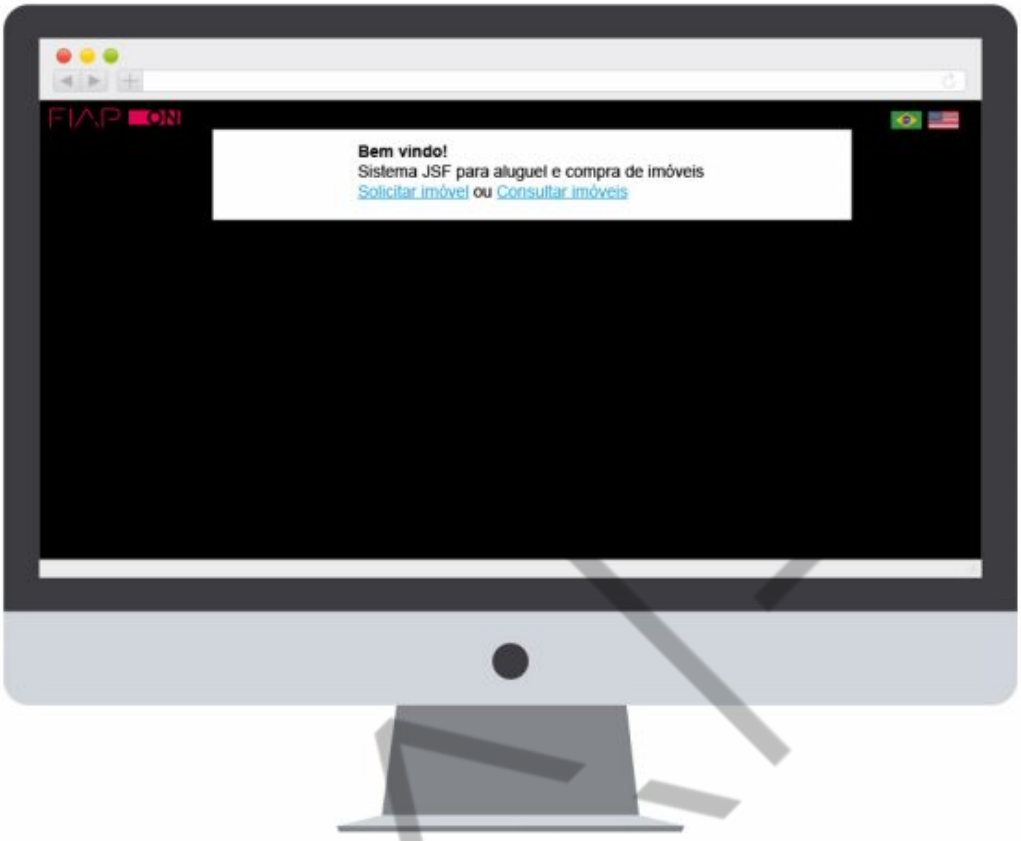

Figura 4.33 – A tela index.xhtml em português
Fonte: Elaborado pelo autor (2018)

Teste a internacionalização, clicando na bandeira norte-americana.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

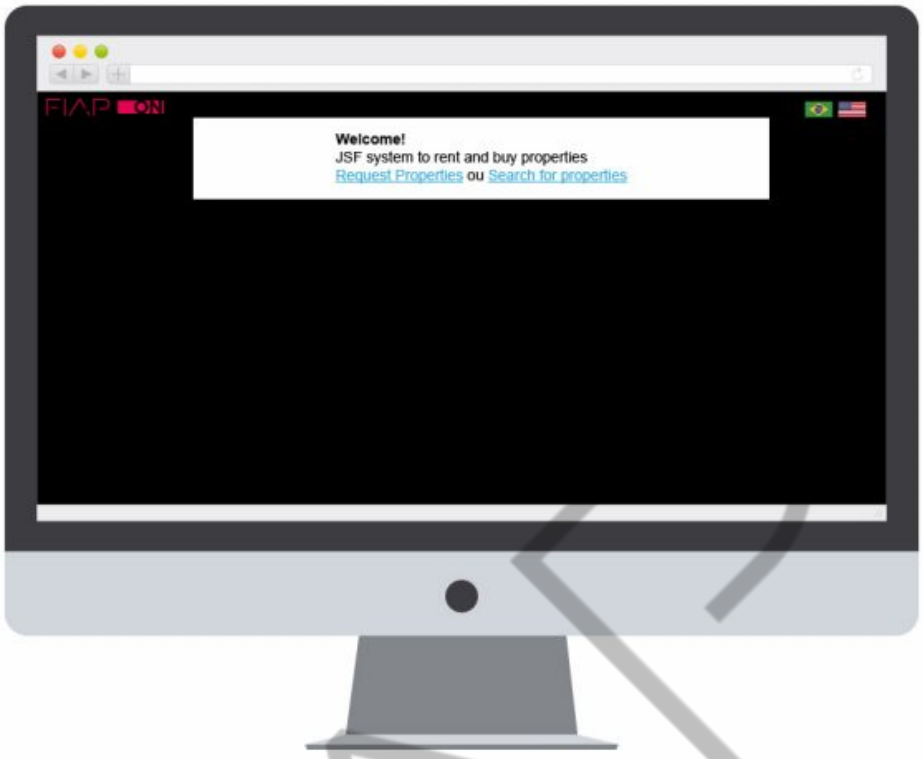

Figura 4.34 – A tela index.xhtml em inglês
Fonte: Elaborado pelo autor (2018)

Navegue para solicitacao.xhtml e perceba que o *Locale* foi mantido na sessão.

Figura 4.35 – A tela solicitacao.xhtml em inglês
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Teste as validações e as mensagens tratadas no arquivo de propriedades, clicando em Enviar.

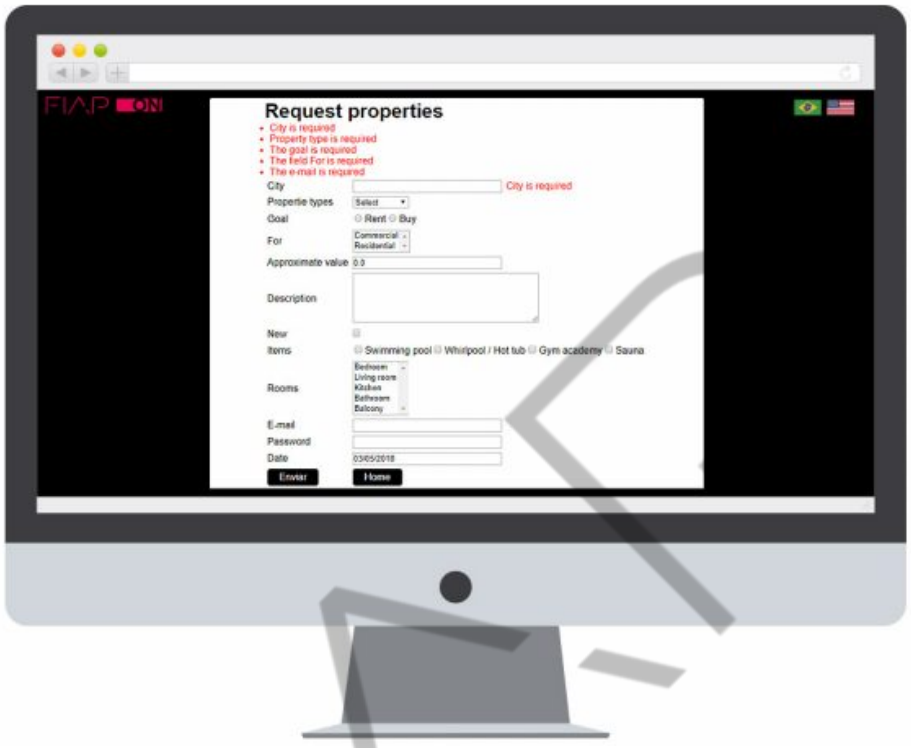

Figura 4.36 – A tela solicitacao.xhtml validada em inglês
Fonte: Elaborado pelo autor (2018)

Retorne para português, preencha todos os campos e faça um teste limpo.

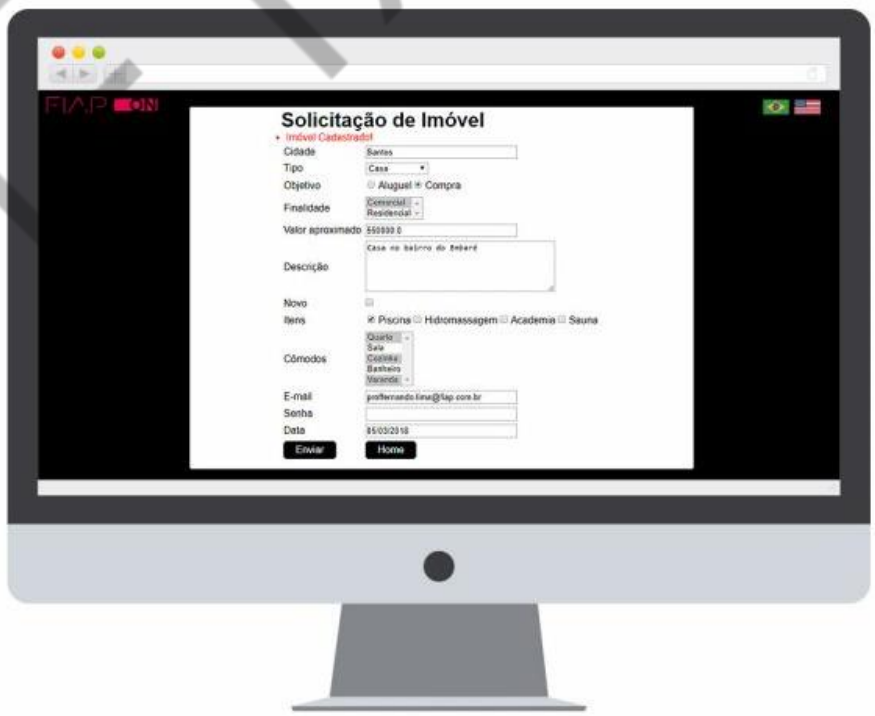

Figura 4.37 – A tela solicitacao.xhtml com cadastro Ok em português
Fonte: Elaborado pelo autor (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.9 Considerações finais

O objetivo deste conteúdo foi abordar o essencial sobre configurações, templates taglibs, componentes, validações, navegação e internacionalização em JSF. Existem diversos pontos a serem explorados, como os diversos componentes das taglibs, validações mais complexas e regras de navegação.

Portanto, sugerimos a leitura dos diversos conteúdos adicionais que são abordados nos livros e URLs de referência deste capítulo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

LUCKOW, D.H.; Melo, A.A. **Programação Java para Web**. São Paulo: Novatec, 2010.

SCHOLTZ, Bauke. **The Definitive Guide to JSF in Java EE 8**: Building Web Applications with JavaServer Faces: Apress, 2018.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

JSF	Java Server Faces
MVC	Model View Controller

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com